

Machine Learning for Biology and Health

CSCI 1851
Spring 2026

Ritambhara Singh

April 07, 2026
Tuesday



Review: Supervised v/s Unsupervised Learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a *function* to map $x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, image captioning, *etc.*

Unsupervised Learning

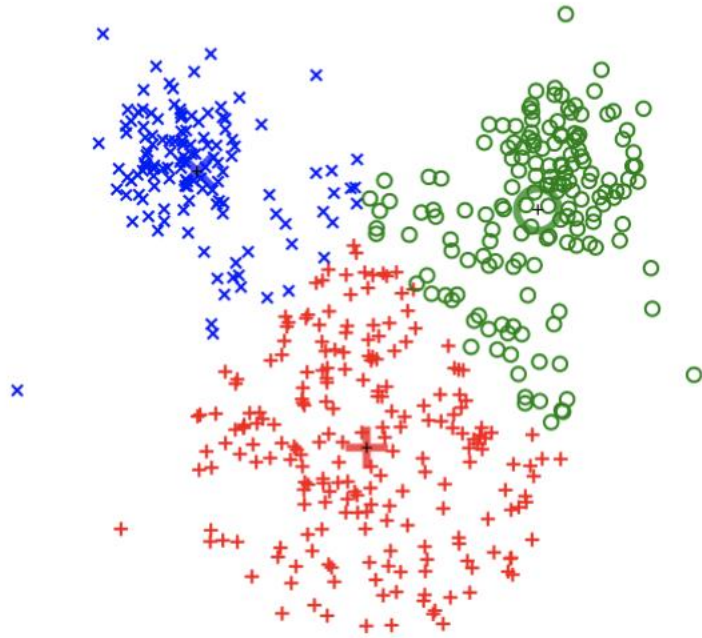
Data: x

Just data, no labels!

Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering, dimensionality reduction, feature learning, density estimation, *etc.*

Review: Unsupervised Learning



k-means clustering

[This image](#) is [CC0 public domain](#)

Unsupervised Learning

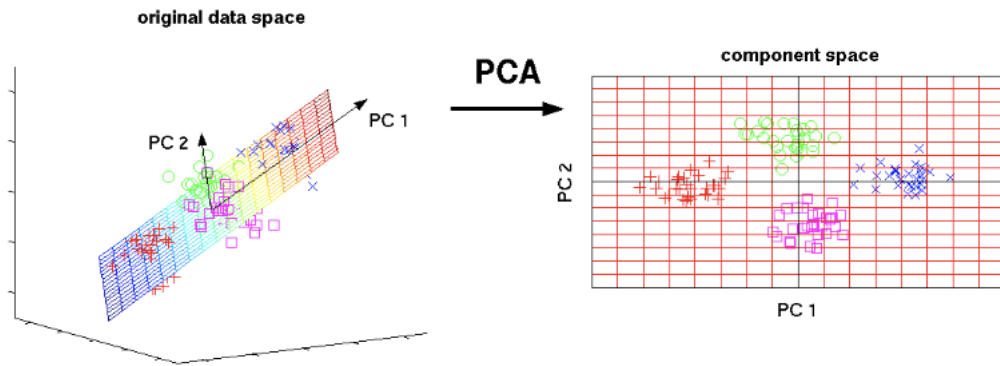
Data: x

Just data, no labels!

Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering,
dimensionality reduction, feature
learning, density estimation, *etc.*

Review: Unsupervised Learning



dimensionality reduction

[This image](#) is [CC0 public domain](#)

Unsupervised Learning

Data: x

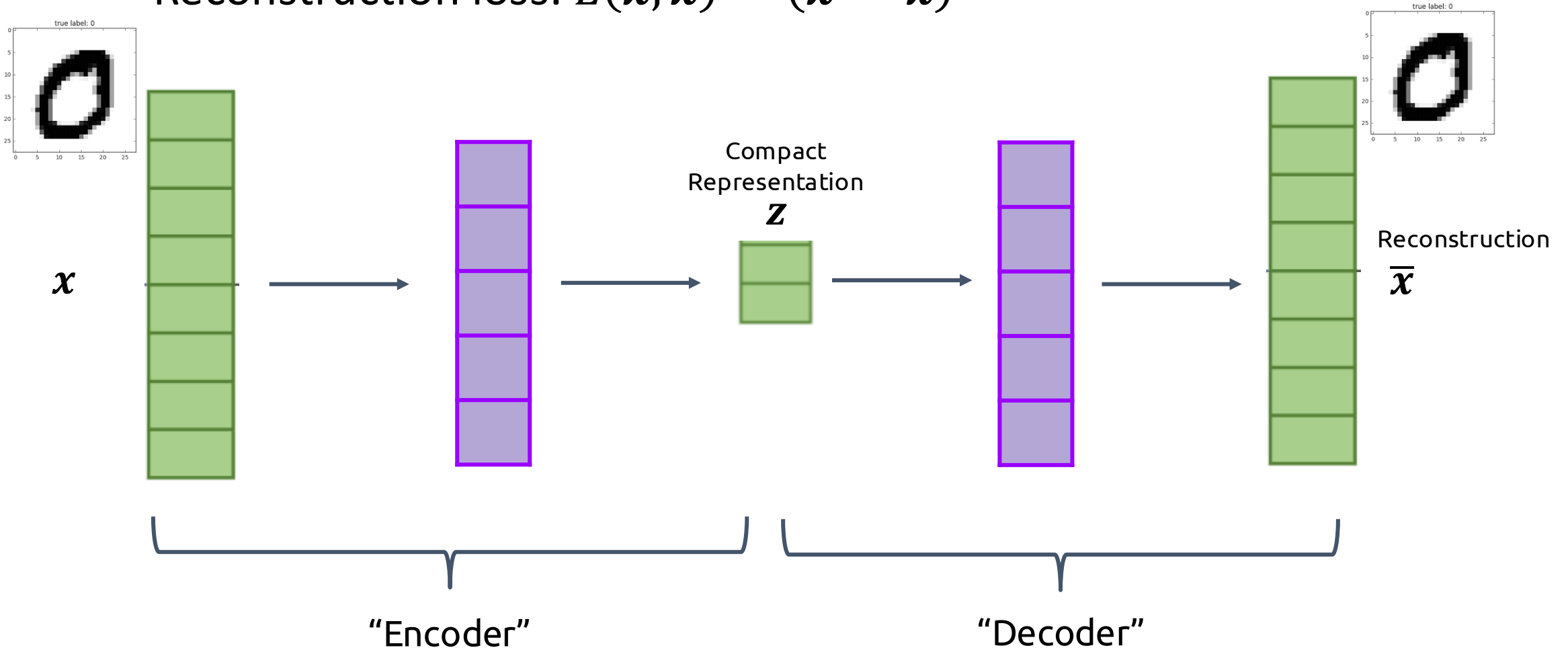
Just data, no labels!

Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering,
dimensionality reduction, feature
learning, density estimation, *etc.*

Review: Autoencoder

- Reconstruction loss: $L(\mathbf{x}, \bar{\mathbf{x}}) = (\mathbf{x} - \bar{\mathbf{x}})^2$



Today's goal – Learn about Variational Autoencoders (VAEs)

(1) Generative modeling

(2) VAE Architecture

(3) VAE Loss

(4) VAE considerations

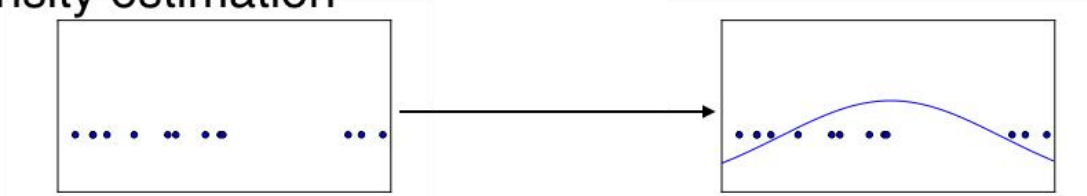
(5) Conditional VAE

(6) Class activity: Designing AEs/VAEs for single-cell datasets

Generative modeling

Unsupervised Learning

- Density estimation



Unsupervised Learning

Data: x

Just data, no labels!

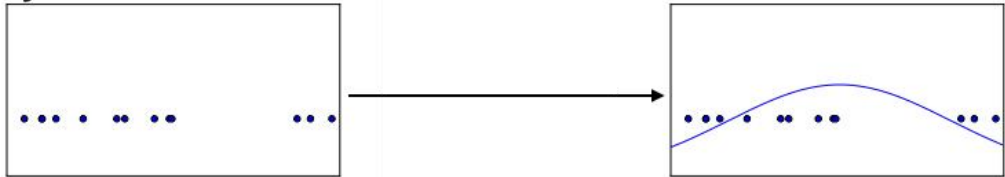
Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering,
dimensionality reduction, feature
learning, density estimation, *etc.*

Unsupervised Learning

Generative models

- Density estimation



- Sample generation



Training examples

Model samples

Unsupervised Learning

Data: x

Just data, no labels!

Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering, dimensionality reduction, feature learning, density estimation, etc.

Discriminative v/s Generative models

Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

Data: x



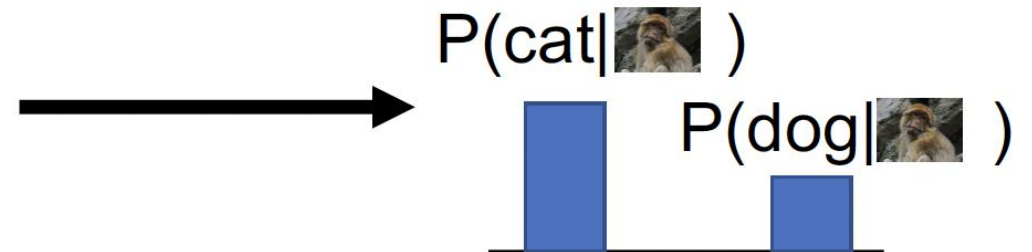
Label: y

Cat

Discriminative v/s Generative models

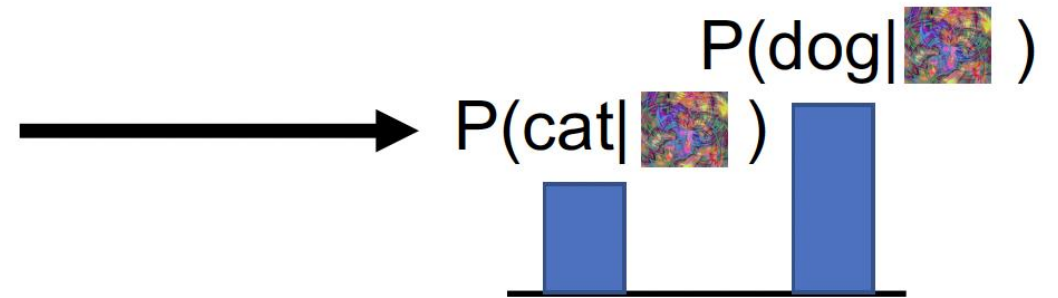
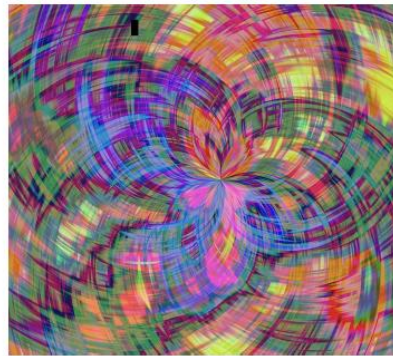
Discriminative Model:

Learn a probability distribution $p(y|x)$



Generative Model:

Learn a probability distribution $p(x)$



Discriminative model: No way for the model to handle unreasonable inputs; it must give label distributions for all images

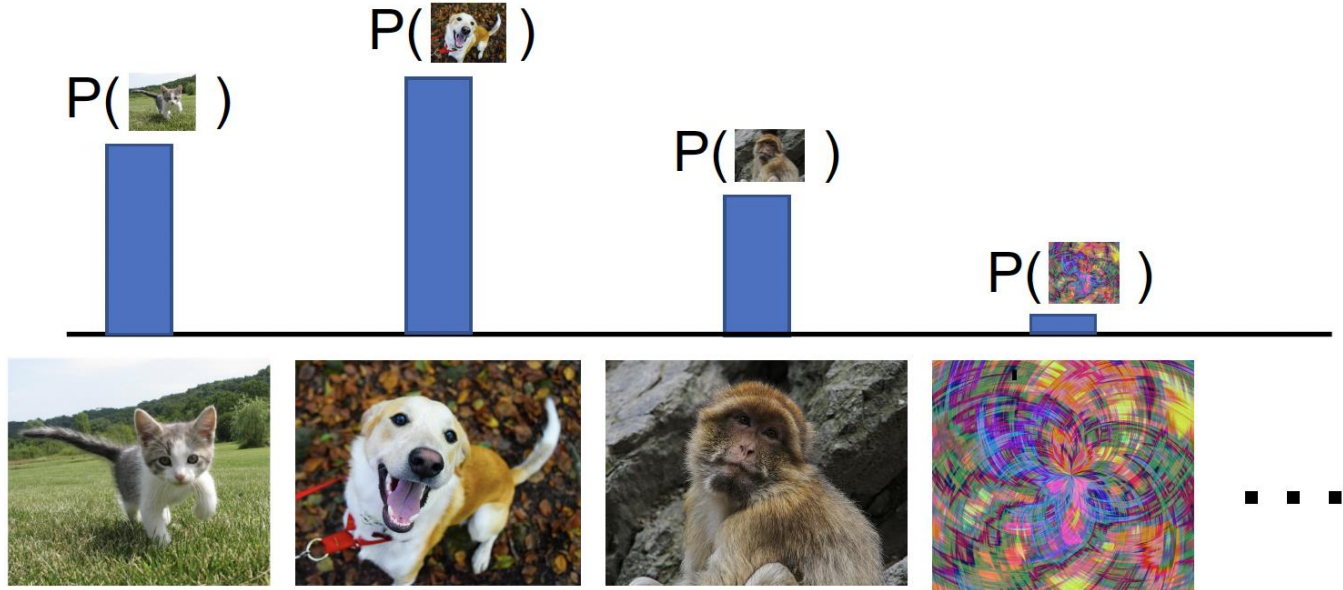
Discriminative v/s Generative models

Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

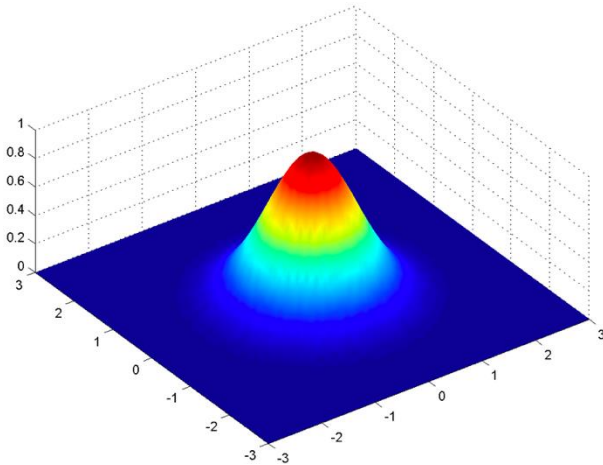
Learn a probability distribution $p(x)$



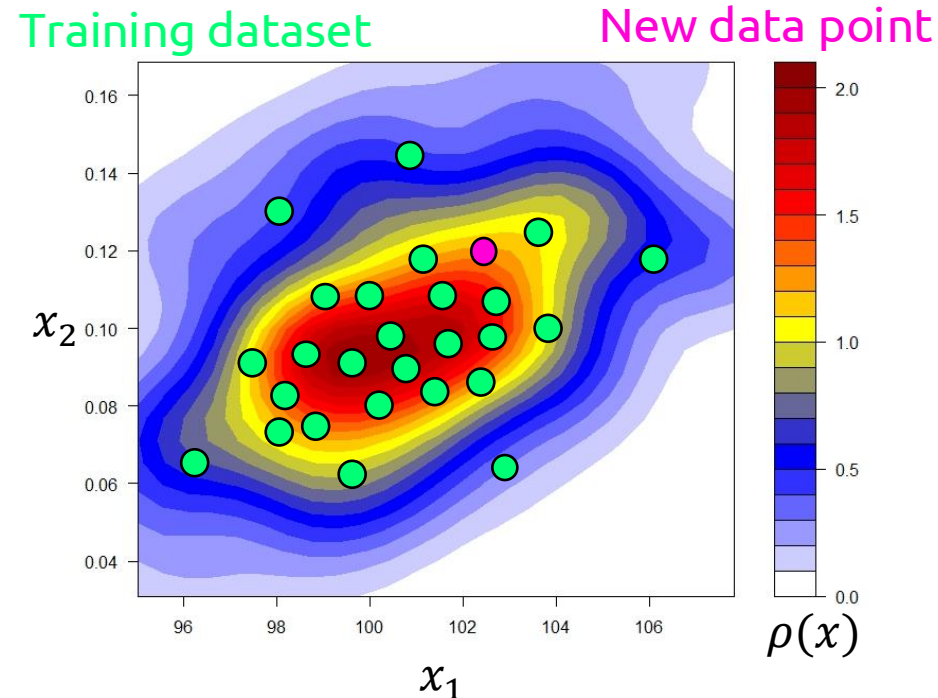
- Generative model: All possible images compete with each other for probability mass
- Intuition: Generation should require deep understanding! Is a dog more likely to sit or stand? How about 3-legged dog vs 3-armed monkey?
- Model can “reject” unreasonable inputs by assigning them small values

Generative Modeling Is:

1. A procedure for (approximately) **sampling** from the distribution from which a dataset was drawn



$$x' \sim p(\cdot)$$



*Probability density is the relationship between observations and their probability.

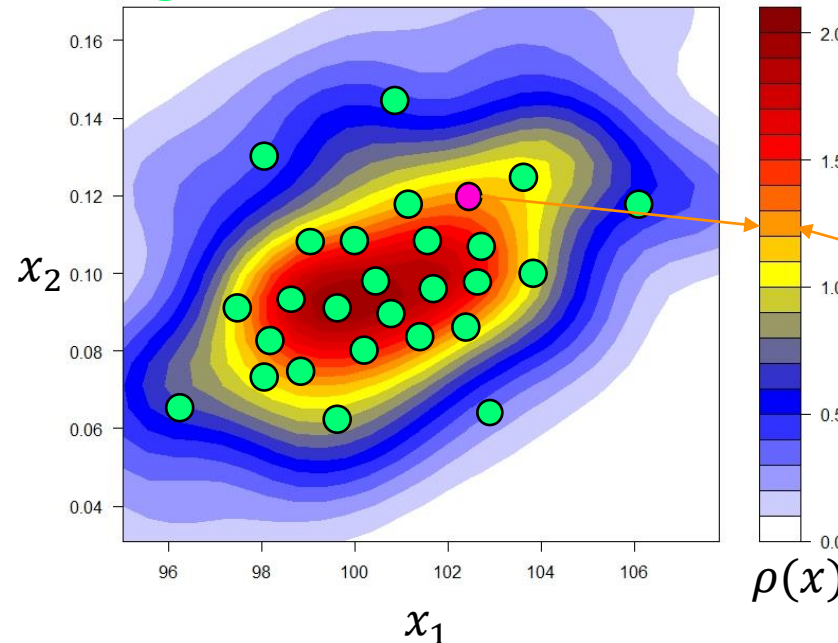
Generative Modeling:

1. A procedure for (approximately) **sampling** from the distribution from which a dataset was drawn
2. A procedure for (approximately) **evaluating the probability density** of a datapoint under the distribution from which a dataset was drawn

$$x' \sim p(\cdot)$$

Training dataset

New data point



$$\rho(x')$$

*Probability density is the relationship between observations and their probability.

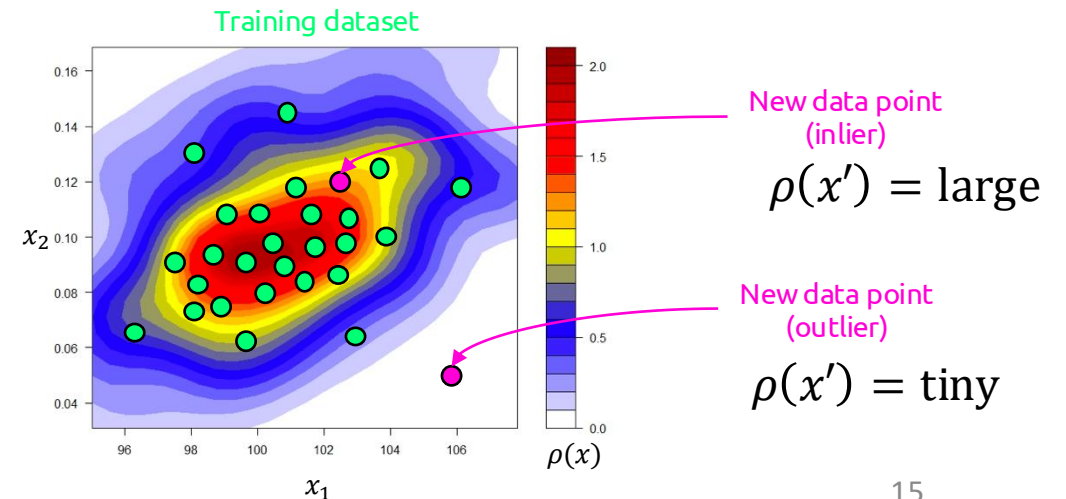
These two views are both useful

1. A procedure for (approximately) **sampling** from the distribution from which a dataset was drawn
2. A procedure for (approximately) **evaluating the probability density** of a datapoint under the distribution from which a dataset was drawn

Application: visual creativity



Application: outlier detection



What are some example generative models?

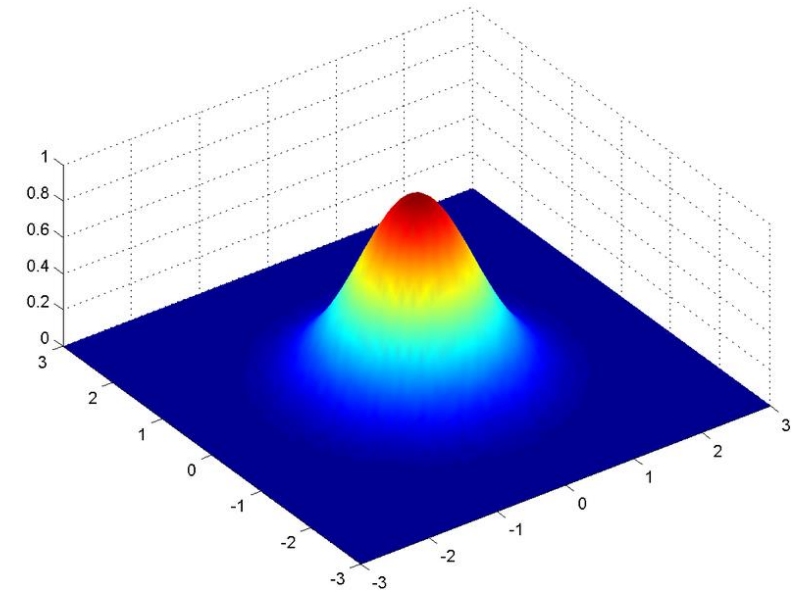
- Any probability distribution can be a generative model
- You already know some of these!

- E.g. The Gaussian Distribution

- $p(x \mid \mu, \sigma) = \mathcal{N}(\mu, \sigma)(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$

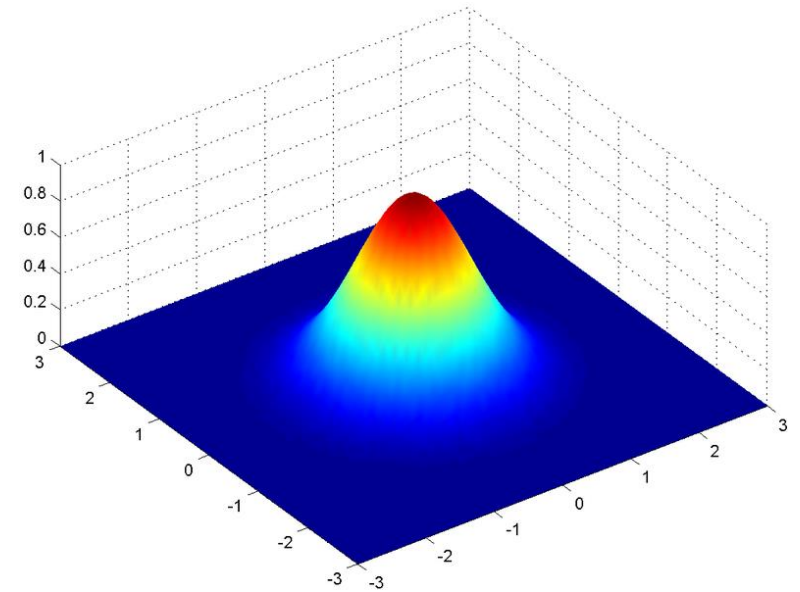
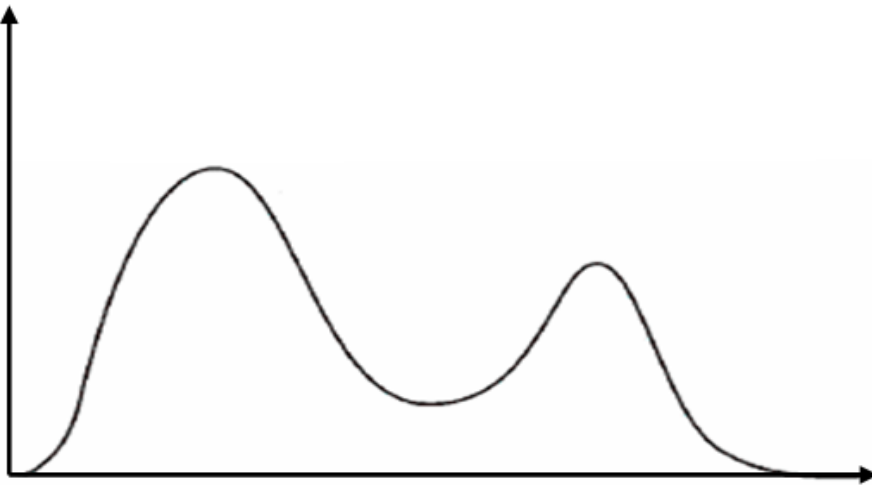
- Sampling:

- [Sample from the unit normal distribution](#) $\rightarrow r \sim \mathcal{N}(0, 1)$
 - Return $\mu + r\sigma$



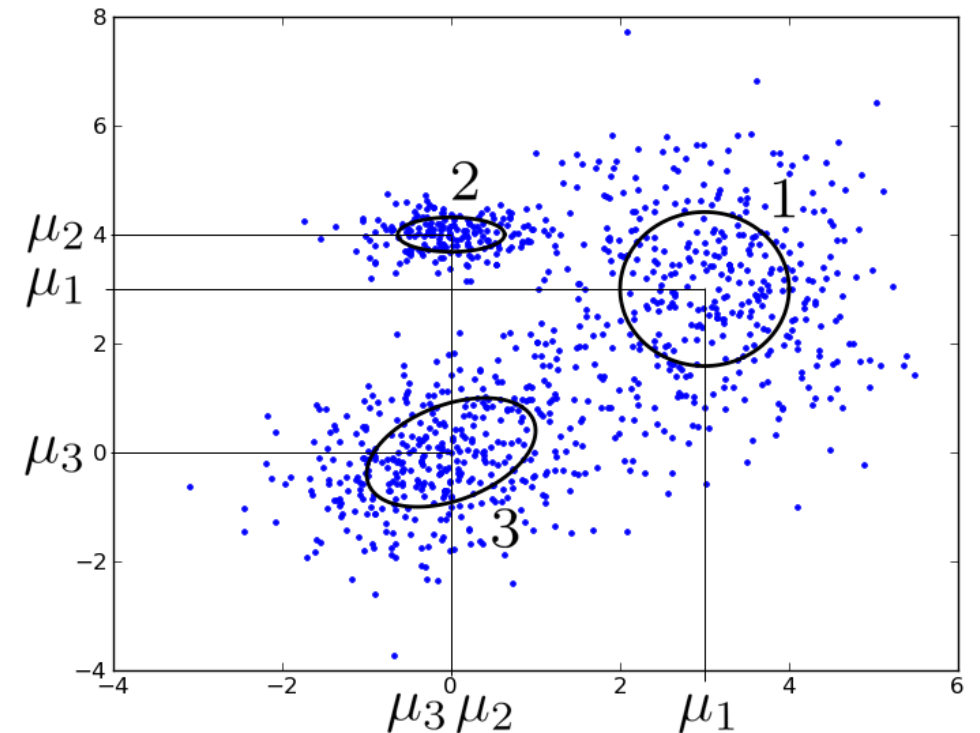
Disadvantages of Gaussian distribution

- Can only represent distributions with a single ***mode***
 - What if the distribution has multiple “peaks?”
 - E.g. book prices (concentrates around different price points if it's hardcover, paperback, e-book, ...)

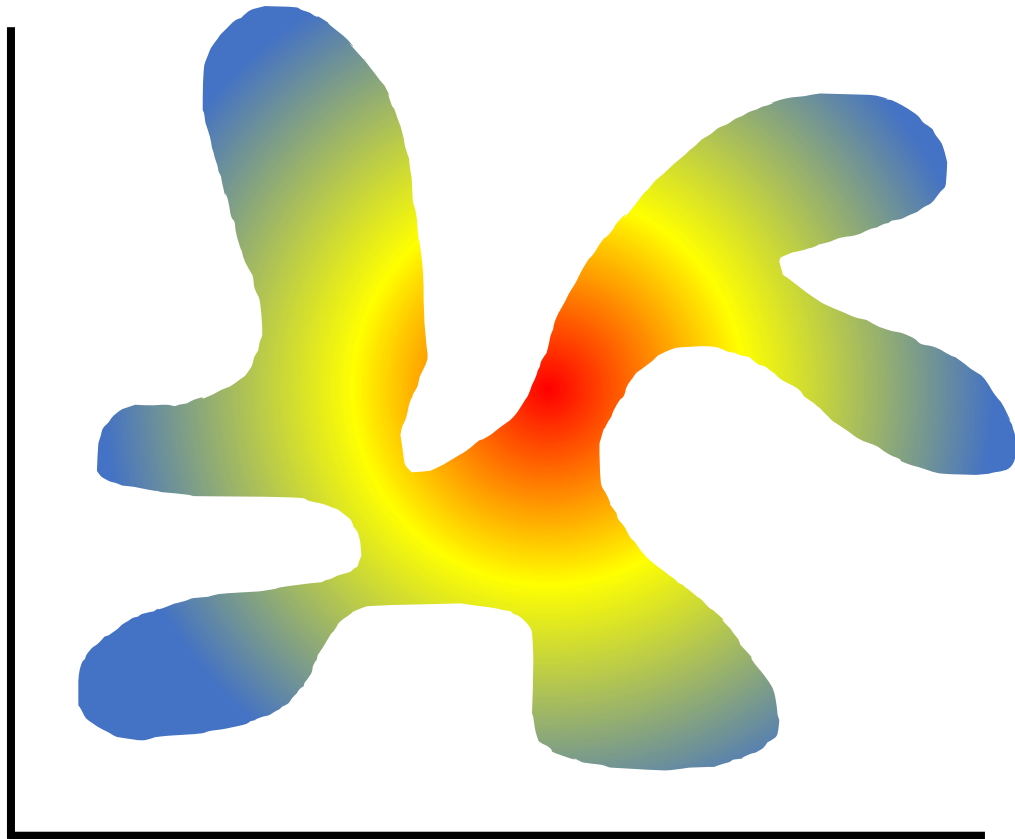


Better: *Mixture* of Gaussians

- A linear combination of multiple individual Gaussian distributions
 - $p(x | w, \mu, \sigma) = \sum_i w_i \mathcal{N}(\mu_i, \sigma_i)(x)$
 - Sampling:
 - Sample from the discrete weight distribution w to choose a Gaussian
 - Sample from that Gaussian as before

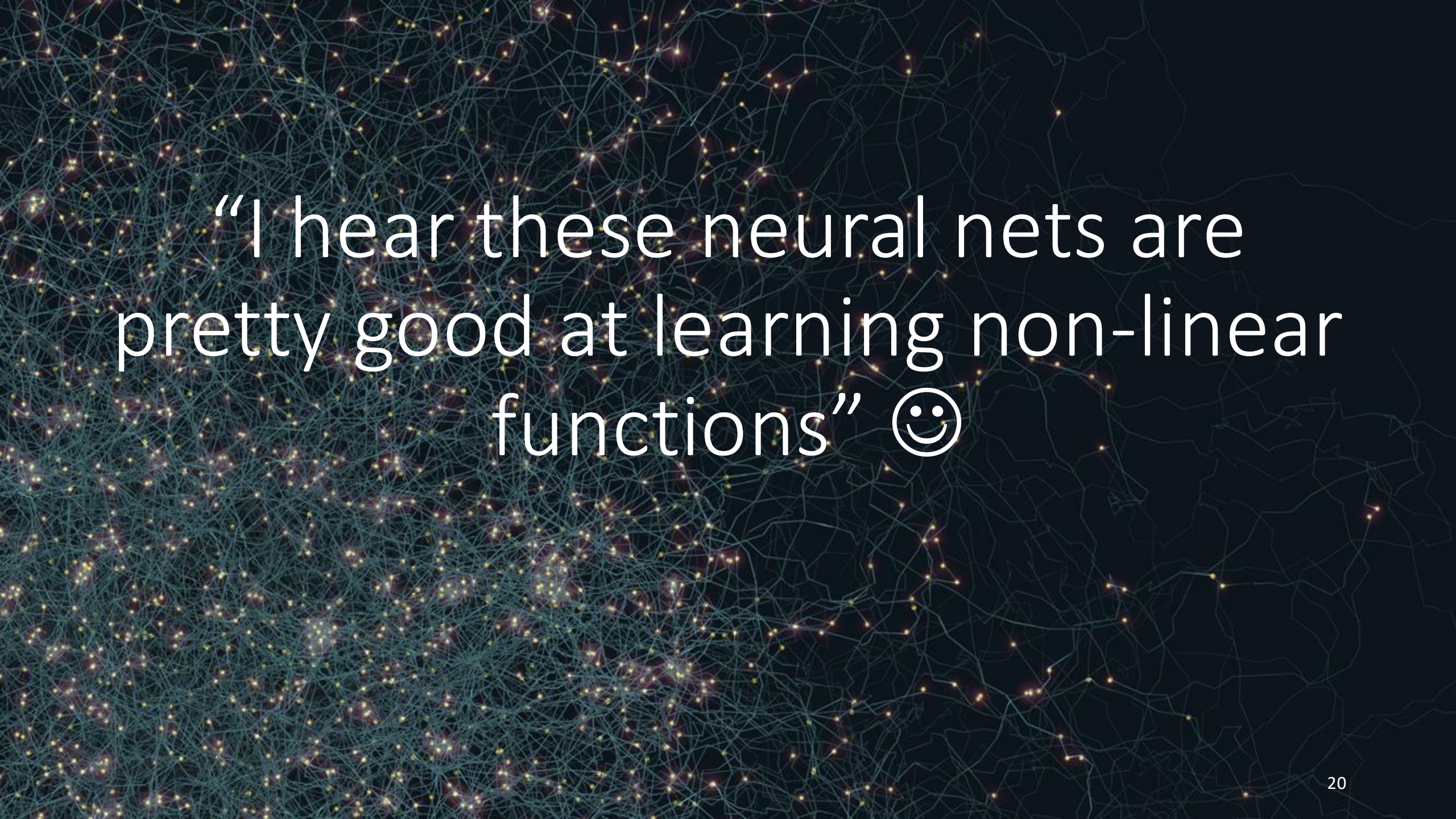


What about something like this?



- This doesn't look like a linear combination of Gaussians...
- ...but maybe it can be expressed as a ***nonlinear*** function of Gaussians?

What can we do?

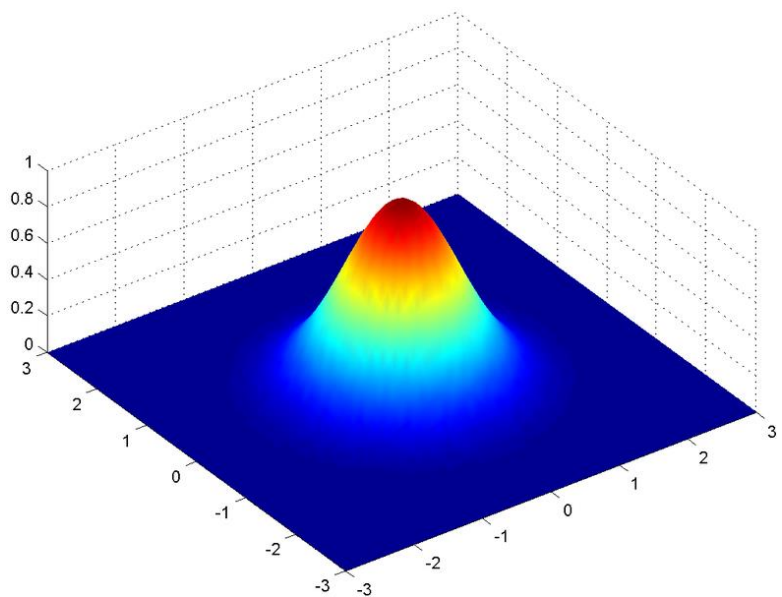


“I hear these neural nets are
pretty good at learning non-linear
functions” 😊

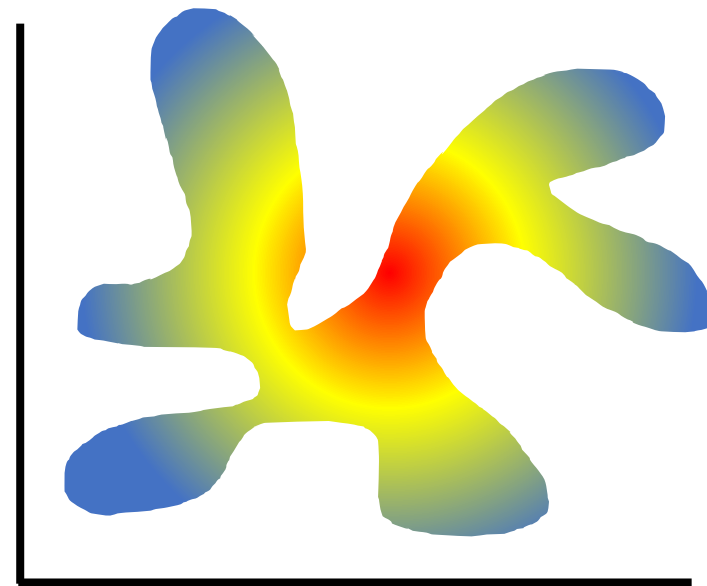


A Neural Generative Model

- Input: a point $z \in \mathbb{R}^n$ drawn from a normal distribution $\mathcal{N}(\mu, \sigma)$
- Output: a point $x \in \mathbb{R}^m$ distributed according to some more complex distribution

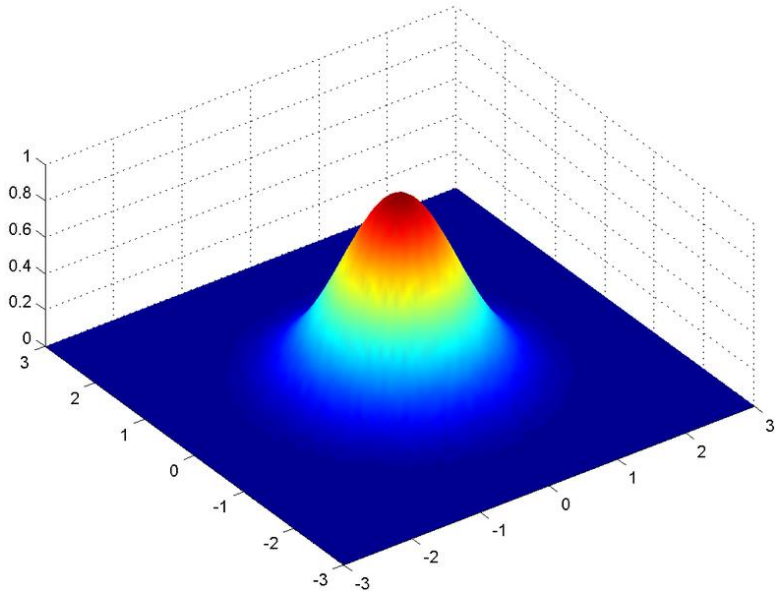


Generator
Network

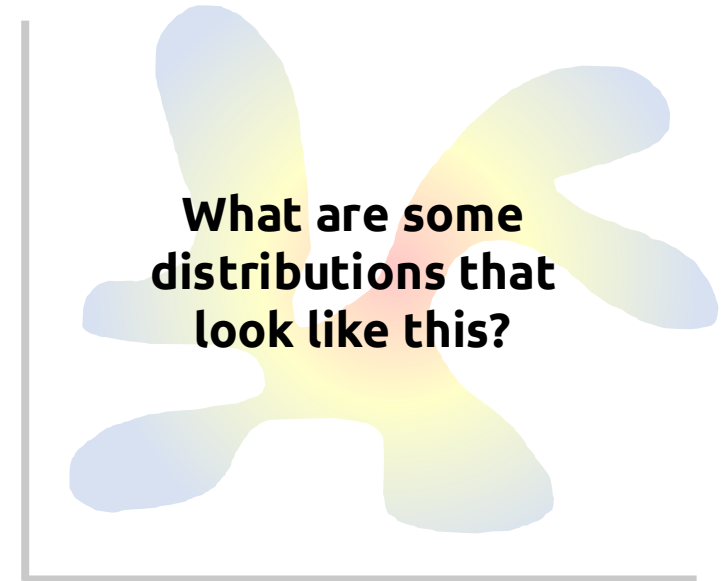


A Neural Generative Model

- Input: a point $z \in \mathbb{R}^n$ drawn from a normal distribution $\mathcal{N}(\mu, \sigma)$
- Output: a point $x \in \mathbb{R}^m$ distributed according to some more complex distribution

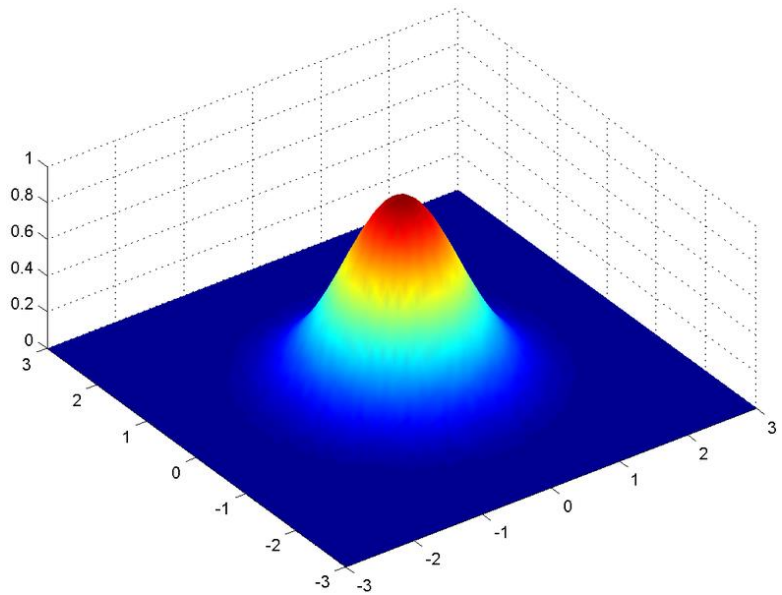


Generator
Network

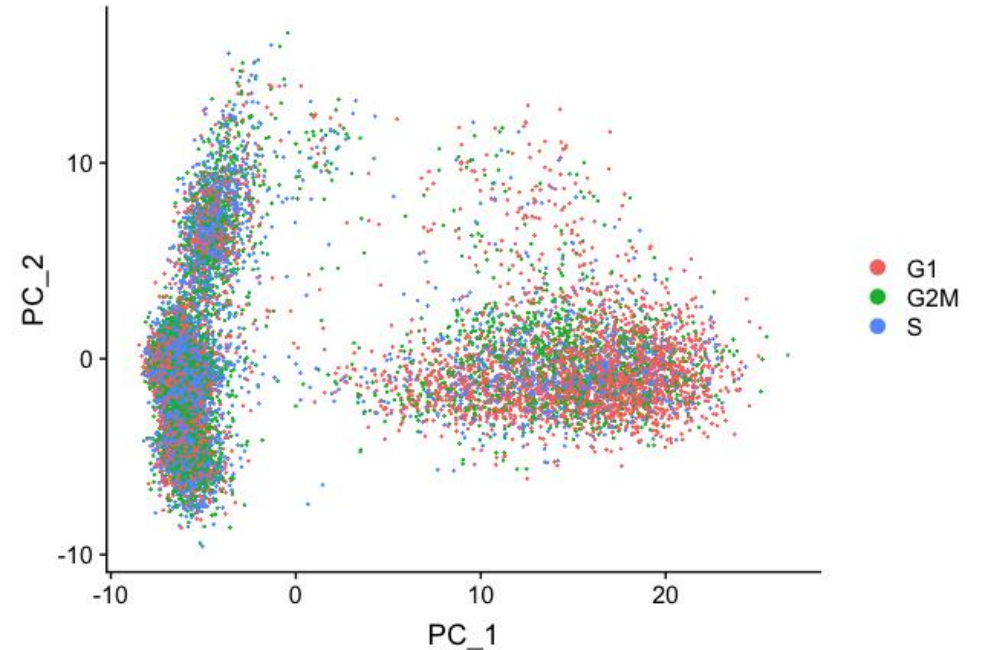


A Neural Generative Model

- Input: a point $z \in \mathbb{R}^n$ drawn from a normal distribution $\mathcal{N}(\mu, \sigma)$
- Output: a point $x \in \mathbb{R}^m$ distributed according to some more complex distribution

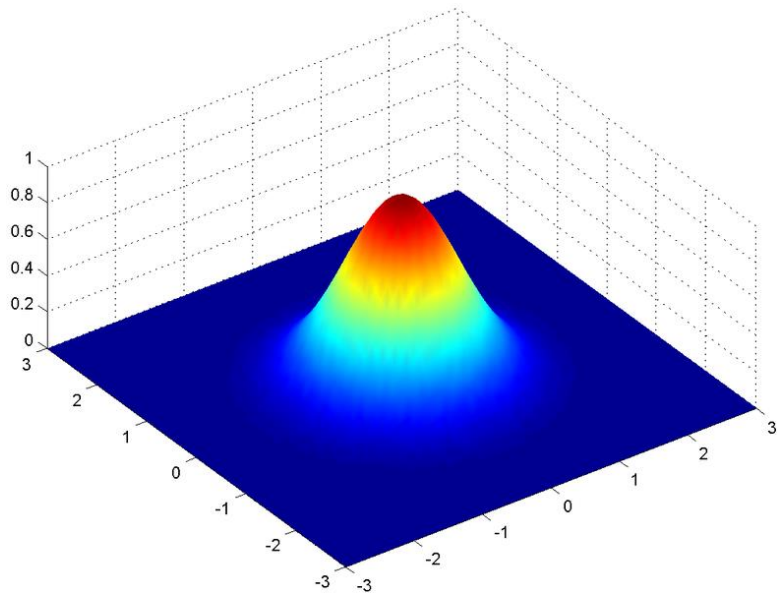


Generator
Network

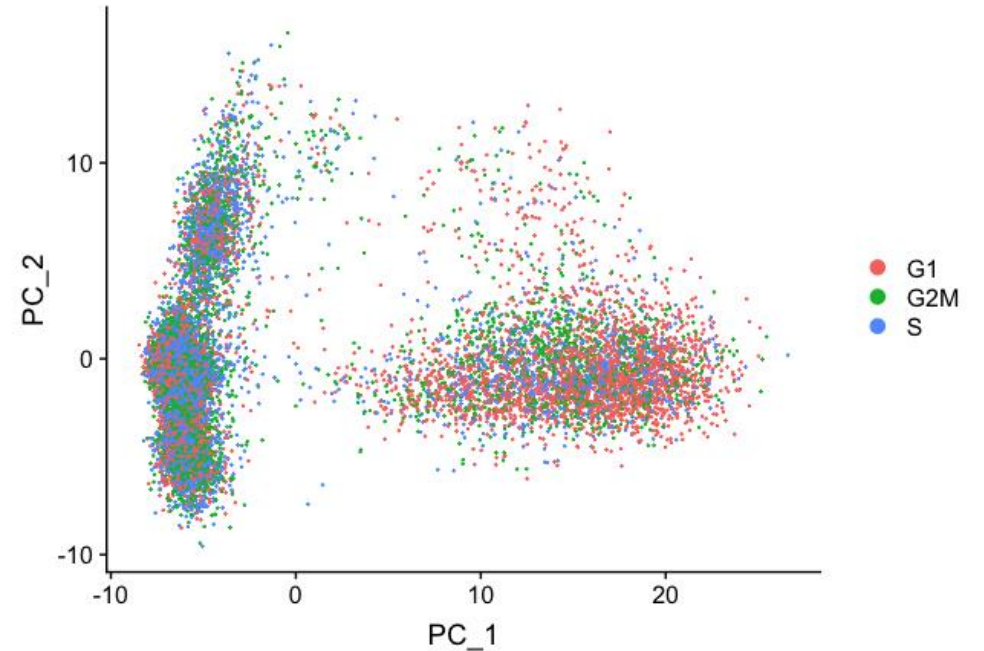


A Neural Generative Model

- Great! So...how do we train this thing?
 - Let's modify our autoencoder to achieve this



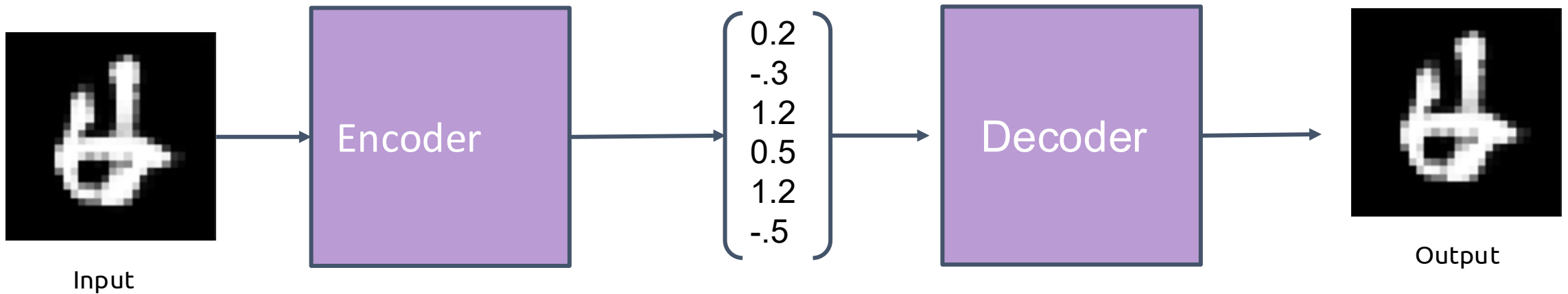
Generator
Network



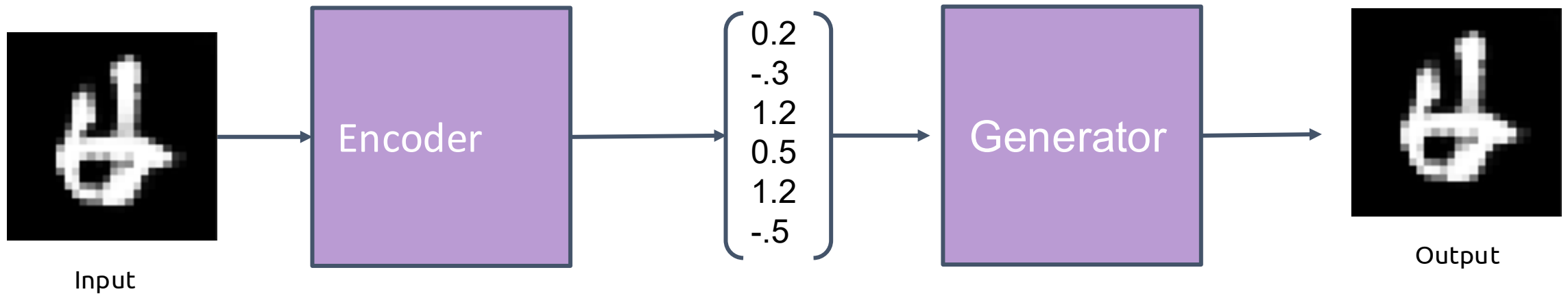
Variational Autoencoder (VAE) Architecture

Autoencoder

Let's think for a bit – how to modify the autoencoder to make it a generative model?

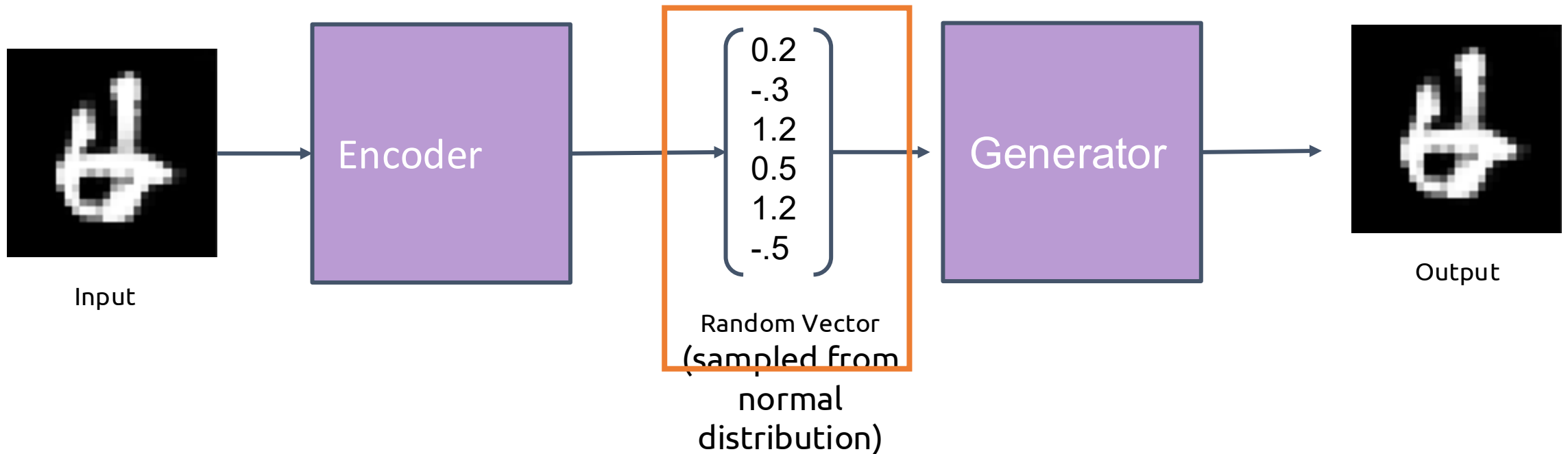


Variational Autoencoders (VAEs)



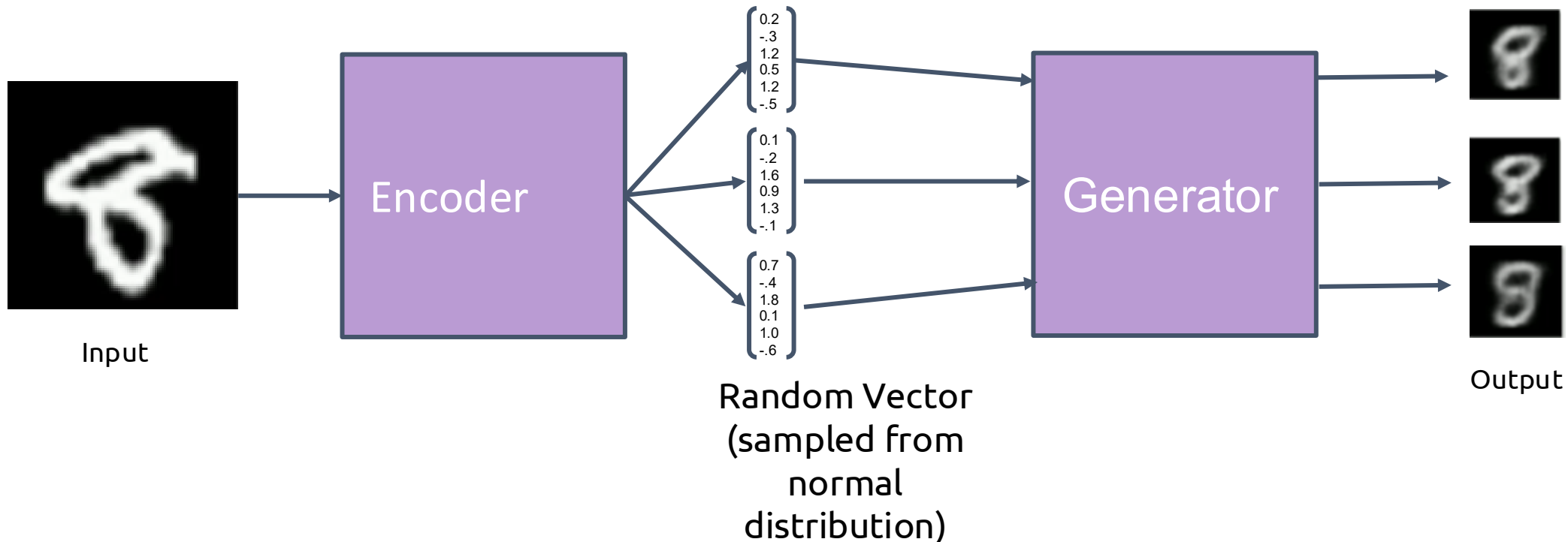
Variational Autoencoders (VAEs)

- This looks almost exactly like an autoencoder...
- ...except that this bottleneck vector is randomly sampled
 - We'll see how in a few slides



Variational Autoencoders (VAEs)

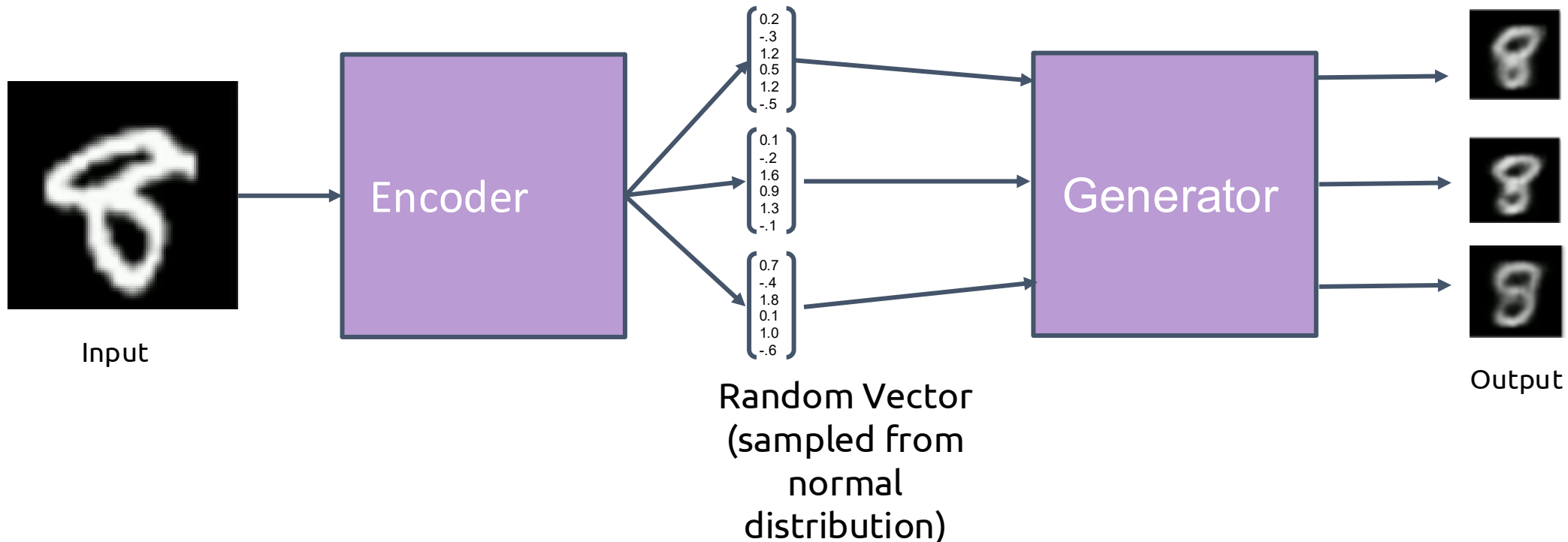
- In fact, the encoder can produce multiple different random vectors...
- ...which then lead to different outputs which are variants of the input



Variational Autoencoders (VAEs)

- ***Why do this?***

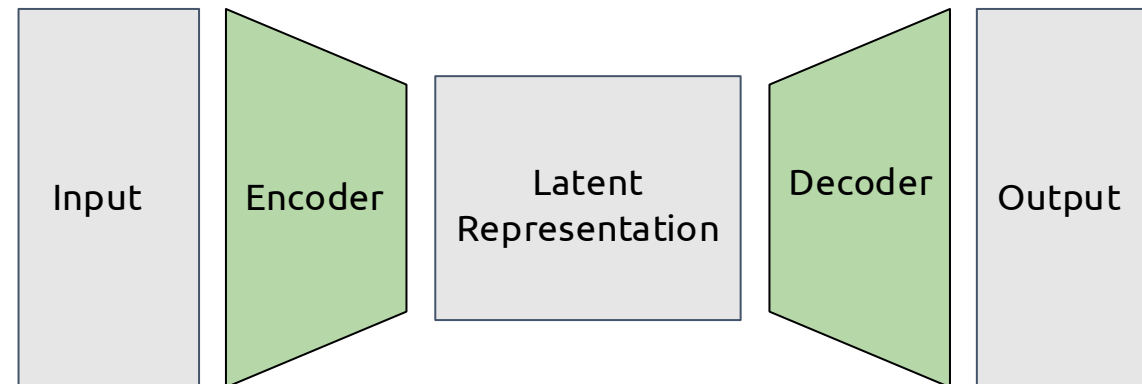
- We'll see shortly how this setup allows for a nice, stable learning algorithm
- (It's actually just a small modification to how autoencoders are trained)



Building up the VAE Architecture

If we were to describe an autoencoder functionally:

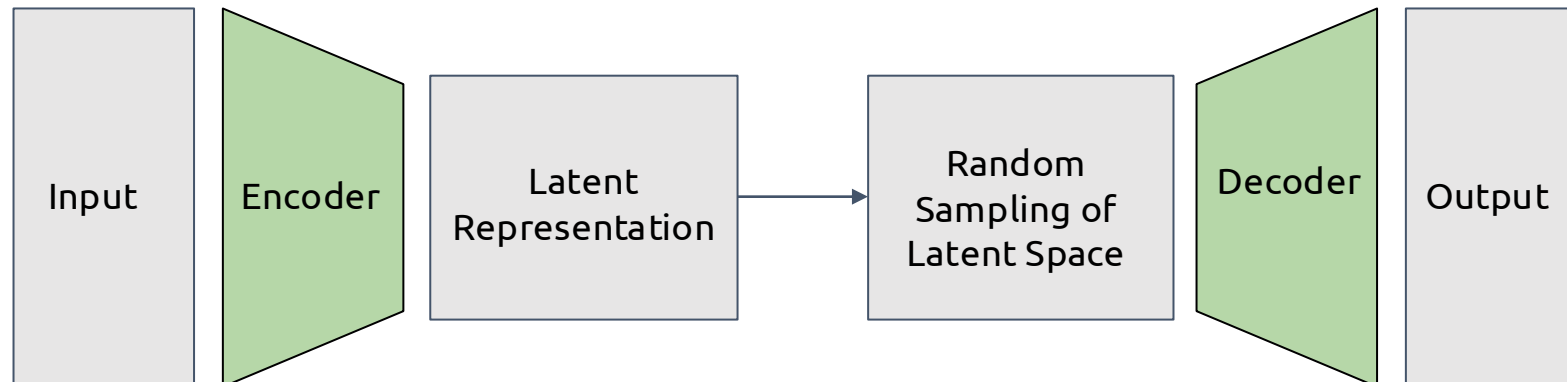
$$\text{Output} = \text{Decoder}(\underbrace{\text{Encoder}(\text{Input})}_{\text{Latent Representation}})$$



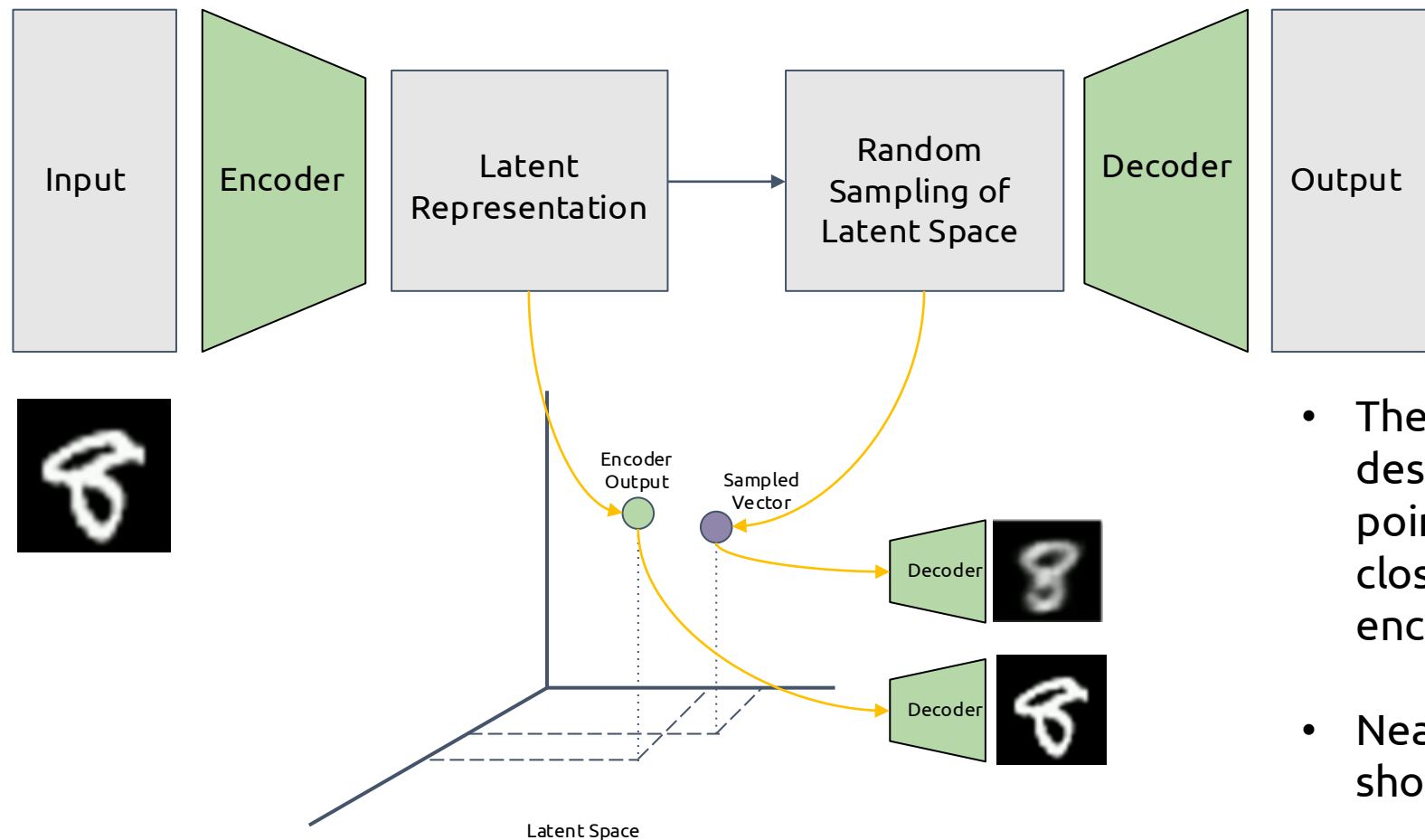
Building up the VAE Architecture

For variational autoencoders, we also do a random sampling operation at the bottleneck

$\text{Output} = \text{Decoder}(\text{random_sample}(\text{Encoder}(\text{Input})))$



How does random sampling in latent space lead to variation?



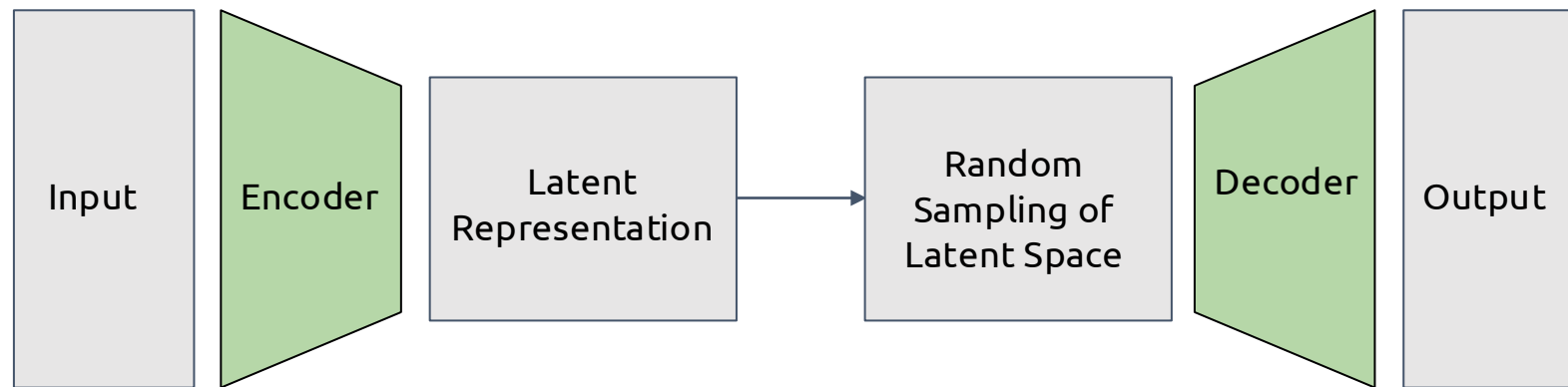
- The random sampling should be designed to produce random points in latent space that are close to the output of the encoder
- Nearby points in the latent space should decode to similar images

How should **random_sample** be defined?

Output = Decoder(**random_sample**(Encoder(Input)))

- We want the sample to be close to the encoder output
- One option: sample from a Gaussian centered at Encoder(Input)

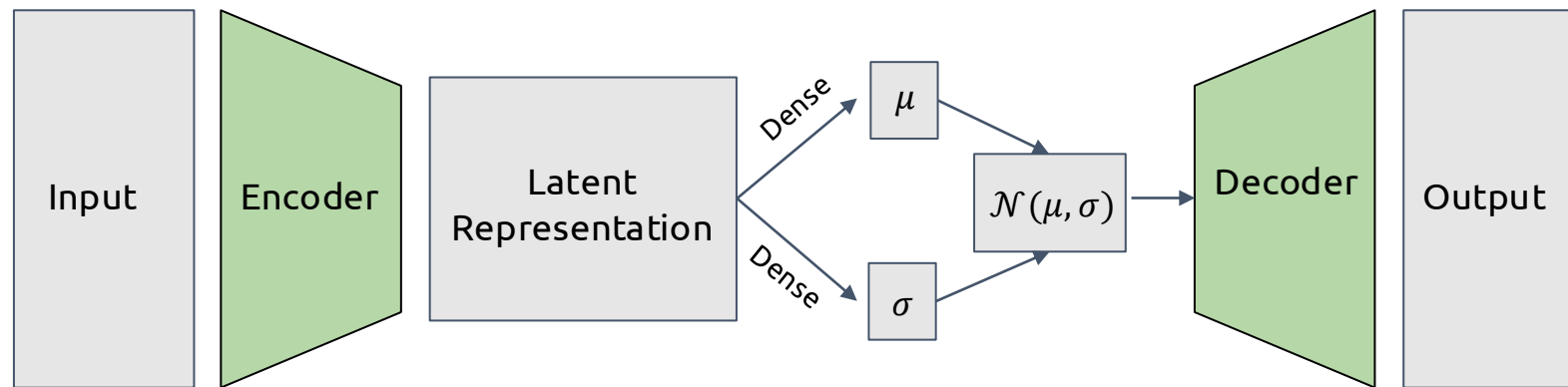
What can we modify?



How should **random_sample** be defined?

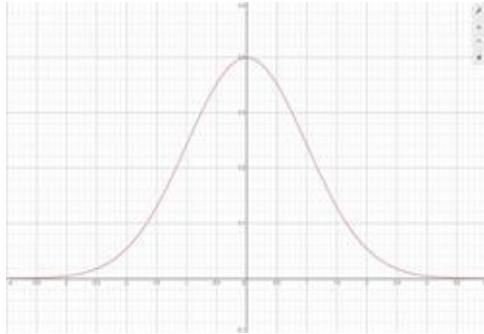
Output = Decoder(**random_sample**(Encoder(Input)))

- We want the sample to be close to the encoder output
- One option: sample from a Gaussian centered at Encoder(Input)
- Use two dense layers to convert the encoder output into the mean and standard deviation of the Gaussian

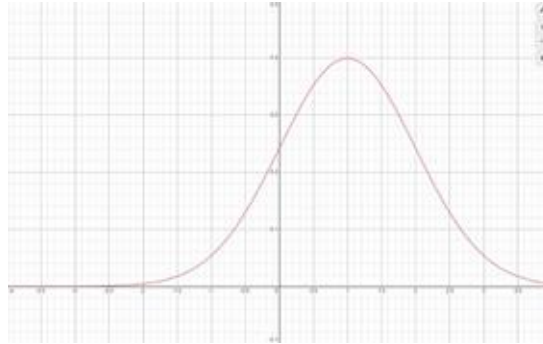




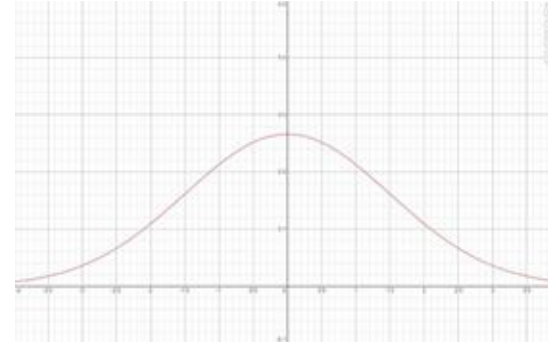
How should **random_sample** be defined?



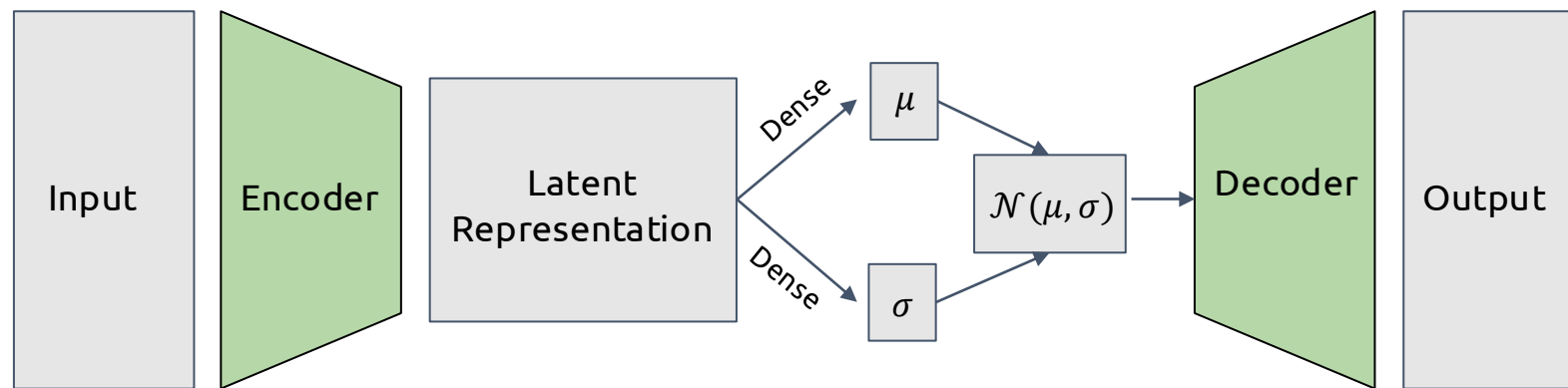
$$\mu = 0$$
$$\sigma = 1$$



$$\mu = 1$$
$$\sigma = 1$$



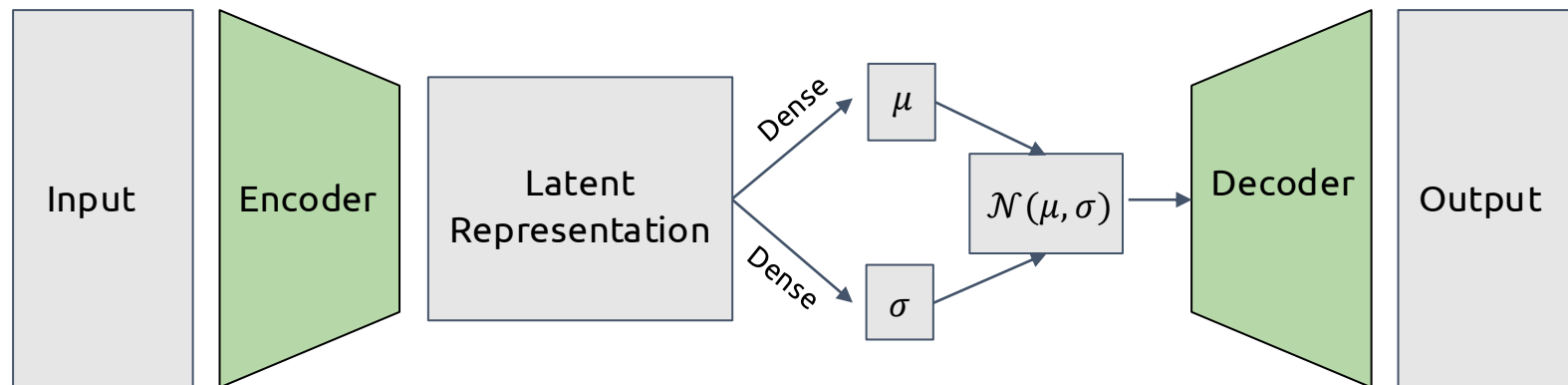
$$\mu = 0$$
$$\sigma = 1.5$$



Training a VAE

Two goals:

1. Reproduce an output similar to the input (Input $\approx$ Output)
 2. Have some variation in our output (Input $\neq$ Output)
- Seems like two conflicting goals!
 - How do we resolve these two goals?



Variational Autoencoder (VAE) Loss

Weighted Combination of Losses

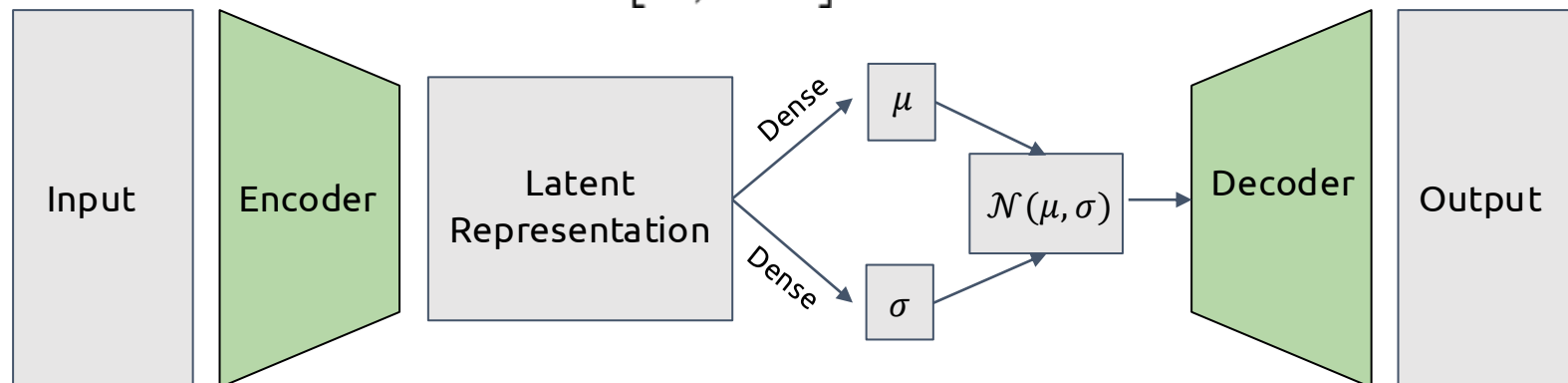
L_1 = loss associated with producing output similar to input

L_2 = loss associated with producing output with some variation to input

$$L = L_1 + \lambda L_2$$

Total Loss:

↑
 $\lambda \in [0, \infty]$



VAE Losses, Defined

We have seen L_1 before: this is just the autoencoder reconstruction loss

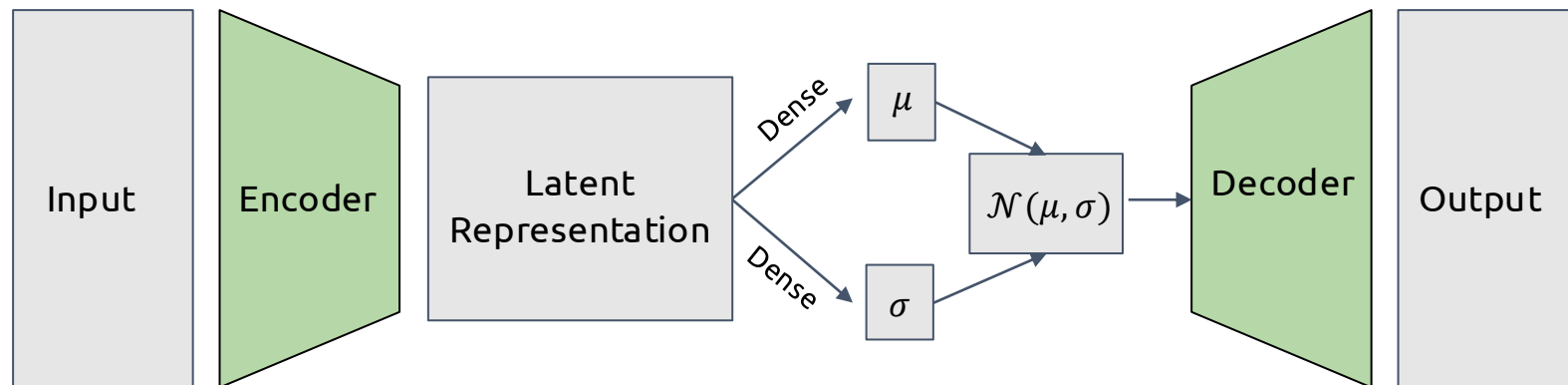
$$L_1(x, \hat{x}) = ||x - \hat{x}||_2^2$$

But with L_2 , it's not so clear. How do we measure how much variation our output would have?

$$L_2(??, ??) = ??$$

Defining the L_2 Loss...

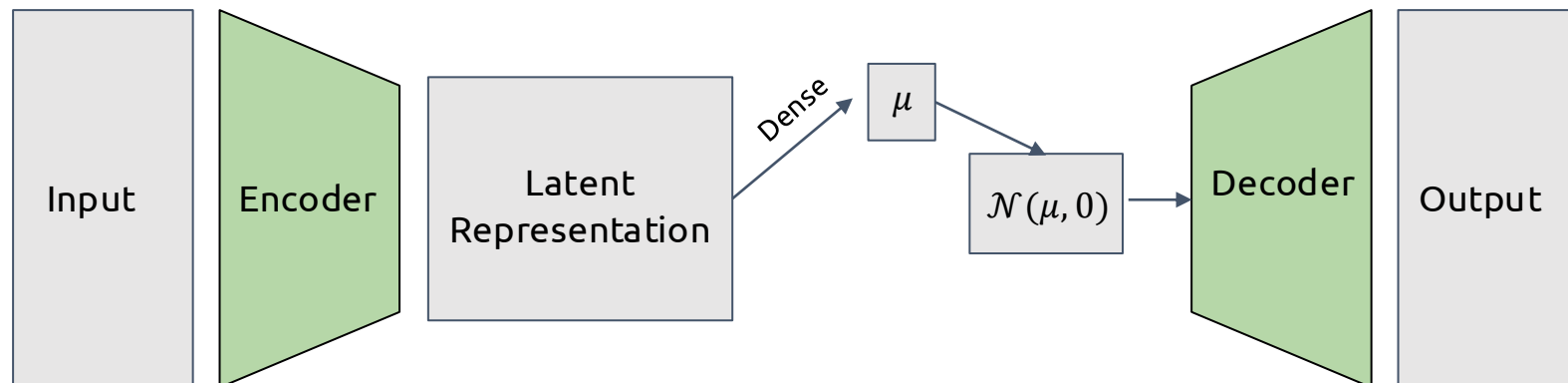
- To get variation, we definitely need a loss that encourages $\sigma > 0$
 - If we don't do this, L_1 will drive σ to zero in an effort to produce the best reconstructions



Defining the L_2 Loss...

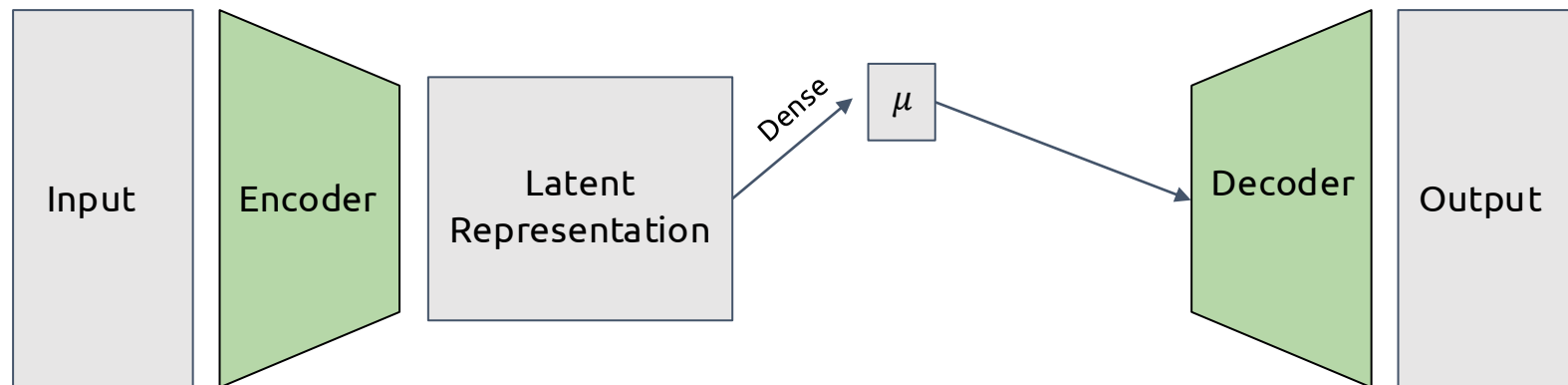
- To get variation, we definitely need a loss that encourages $\sigma > 0$
 - If we don't do this, L_1 will drive σ to zero in an effort to produce the best reconstructions

What's the issue here?



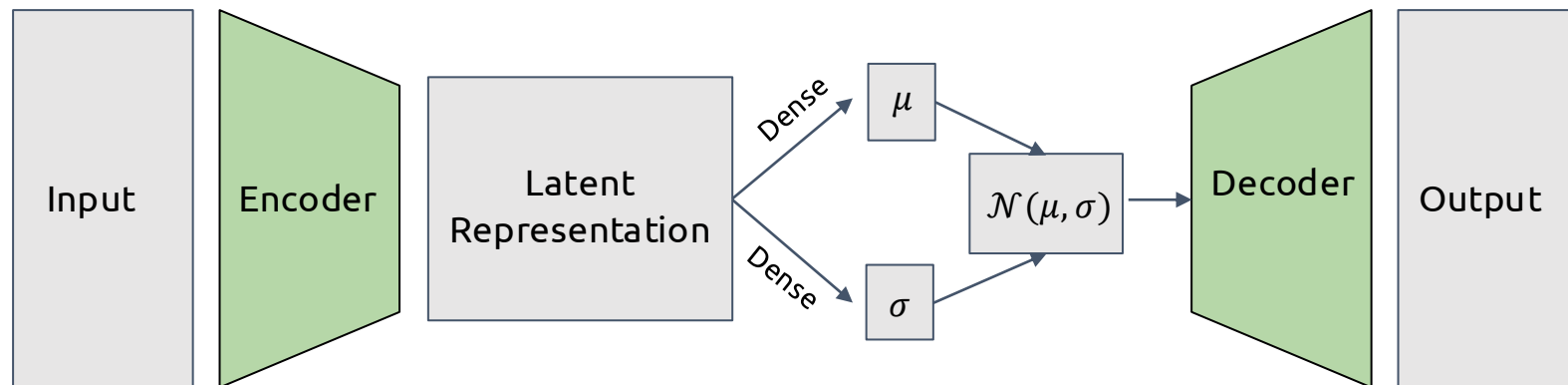
Defining the L_2 Loss...

- To get variation, we definitely need a loss that encourages $\sigma > 0$
 - If we don't do this, L_1 will drive σ to zero in an effort to produce the best reconstructions
 - Behaves the same as a regular autoencoder!



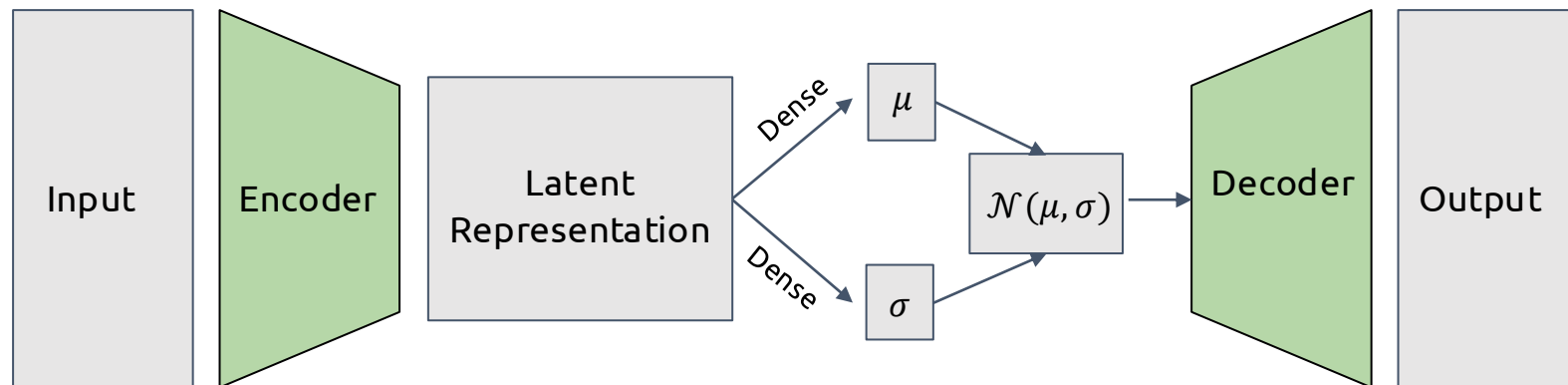
Defining the L_2 Loss...

- To get variation, we definitely need a loss that encourages $\sigma > 0$
 - If we don't do this, L_1 will drive σ to zero in an effort to produce the best reconstructions
- But how big should we encourage σ to be?
- And for that matter, what we do about μ ?



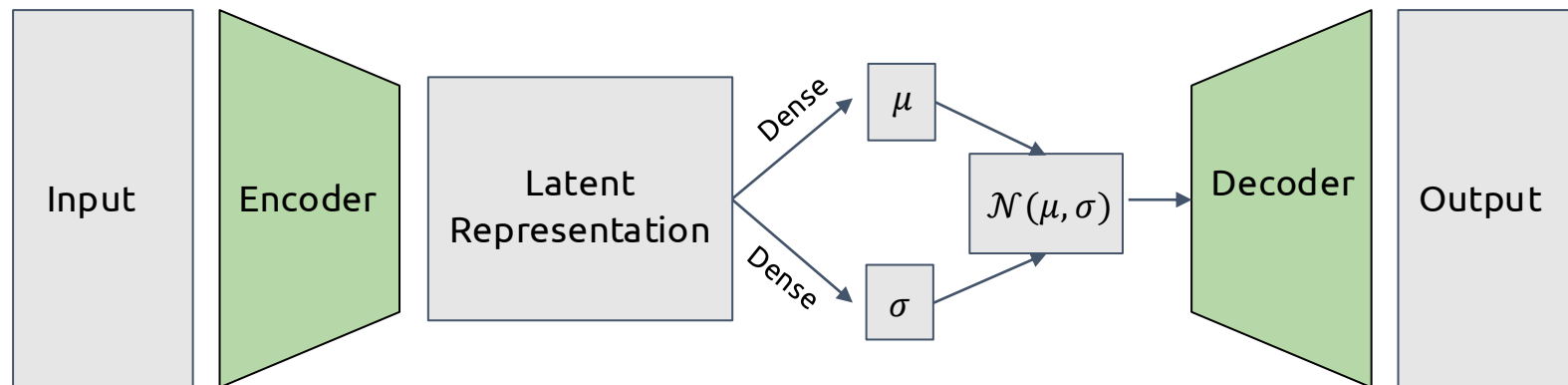
Defining the L_2 Loss...

- **The idea:** make $\mathcal{N}(\mu, \sigma)$ close to $\mathcal{N}(0, 1)$
 - Obviously, we can't perfectly satisfy this for every input (otherwise every input would produce the same set of outputs → terrible reconstruction!)
 - But, we'll see later that having some light pressure to make $\mathcal{N}(\mu, \sigma)$ close to $\mathcal{N}(0, 1)$ will have some beneficial properties



Defining the L_2 Loss...

- Wait...but **how** do we make $\mathcal{N}(\mu, \sigma)$ close to $\mathcal{N}(0, 1)$?
- More generally: how do we measure the difference between two probability distributions?



Kullback–Leibler (KL) Divergence

Measures the difference between any two probability distributions

$$D_{KL}(P||Q) = \int_{-\infty}^{\infty} p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$$

What this says:

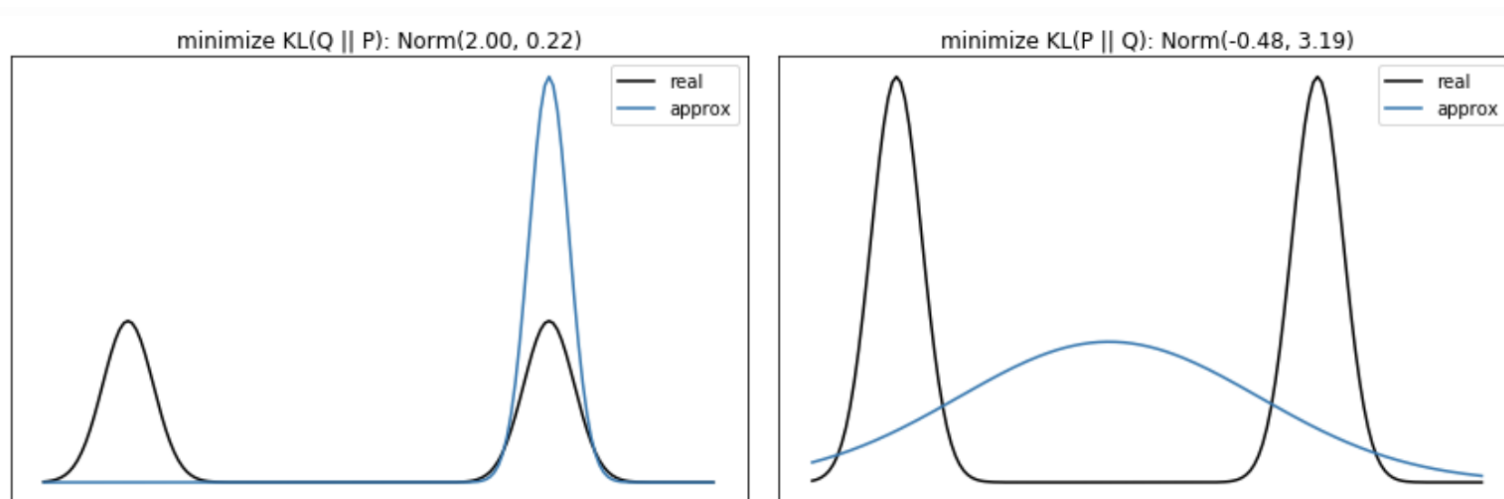
- “Everywhere that p has probability density...”
- “...the difference between p and q should be small”
 - Difference in log probabilities (remember that $\log\left(\frac{a}{b}\right) = \log(a) - \log(b)$)

Kullback–Leibler (KL) Divergence

Measures the difference between any two probability distributions

$$D_{KL}(P||Q) = \int_{-\infty}^{\infty} p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$$

- Note that this is not symmetric: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$



Kullback–Leibler (KL) Divergence

- Expensive to compute, in general (no closed form, have to numerically approximate the integral)
- But! There is a closed form for Gaussians:

$$D_{KL}(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

k is the dimensionality of $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ (e.g. $k = 100$ when $\boldsymbol{\mu} \in \mathbb{R}^{100}$)

We won't derive the equation above, but let's convince ourselves it behaves how we expect it to behave

Kullback–Leibler (KL) Divergence

- Expensive to compute, in general (no closed form, have to numerically approximate the integral)
- But! There is a closed form for Gaussians:

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

Derive the expression for (1) $\sigma=1$ and (2) $\mu=0$

KL Divergence for Two Gaussians

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

Let's take the case $\sigma = 1$

$$\begin{aligned} D_{KL}(\mathcal{N}(\mu, 1) || \mathcal{N}(0, 1)) &= \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + 1^2 - \ln(1) - 1) \\ &= \frac{1}{2} \sum_{i=1}^k \mu_i^2 \end{aligned}$$

The expression is minimized $\mu = 0$ (which is what we want!)

KL Divergence for Two Gaussians

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

Let's take the case $\mu = 0$

$$D_{KL}(\mathcal{N}(0, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\sigma_i^2 - \ln(\sigma_i^2) - 1)$$

This expression is minimized when $\sigma = 1$ (which is also what we want!)

The Final VAE Loss Function

We now have all the tools necessary to construct our loss function.

$$L = L_1 + \lambda L_2 \quad \lambda \in [0, \infty]$$

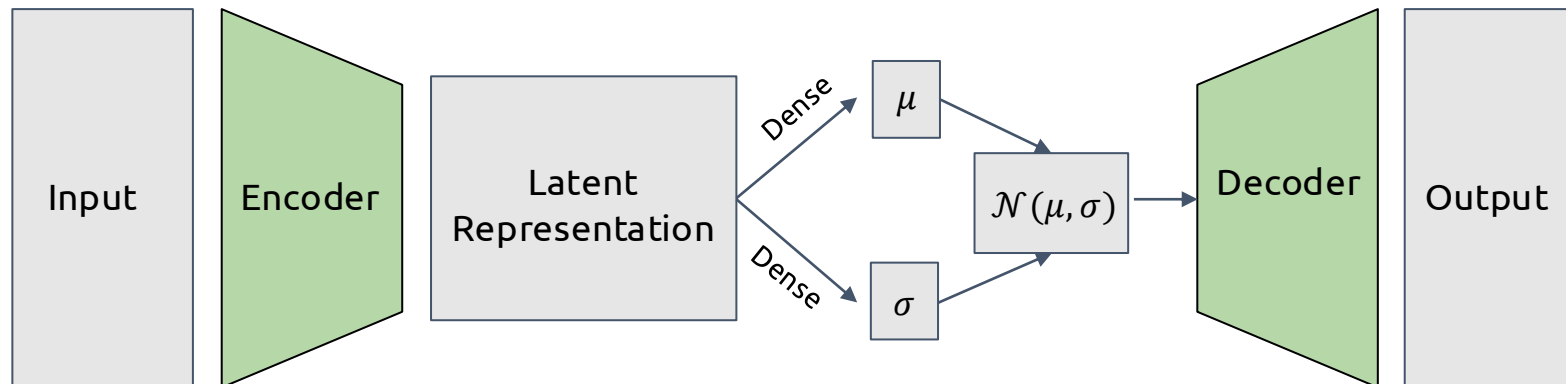
Which turns into this:

$$L = ||x - \hat{x}||_2^2 + \lambda D_{KL}(\mathcal{N}(\mu, \sigma), \mathcal{N}(0, 1))$$



Putting it all together

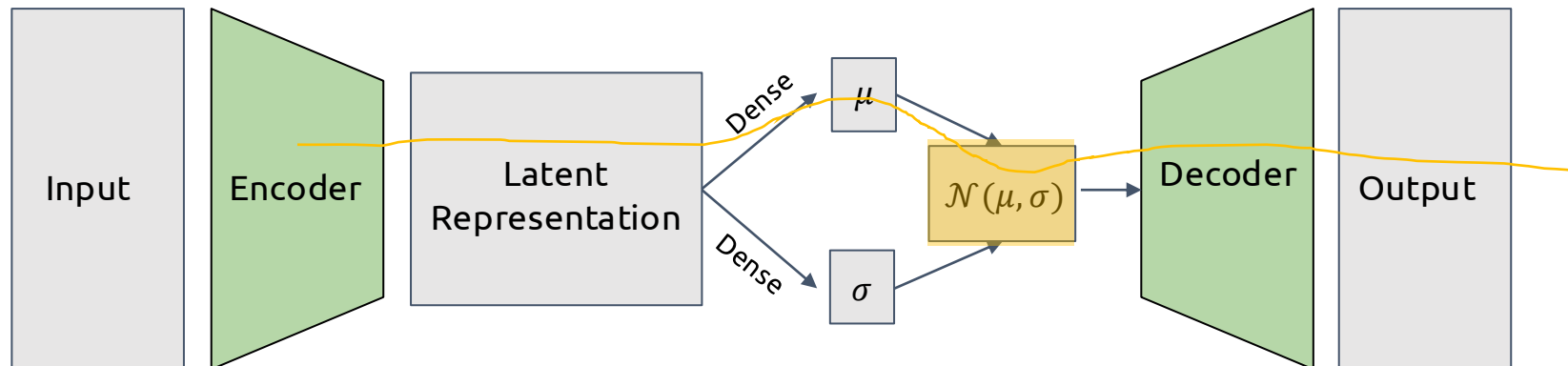
$$L = ||x - \hat{x}||_2^2 + \lambda D_{KL}(\mathcal{N}(\mu, \sigma), \mathcal{N}(0, 1))$$



Ah, but there's a catch:

Can anyone guess?

- To update the weights of the encoder, we have to backprop through a random sampling operation
- Sampling a random value seems not differentiable...



Other VAE considerations

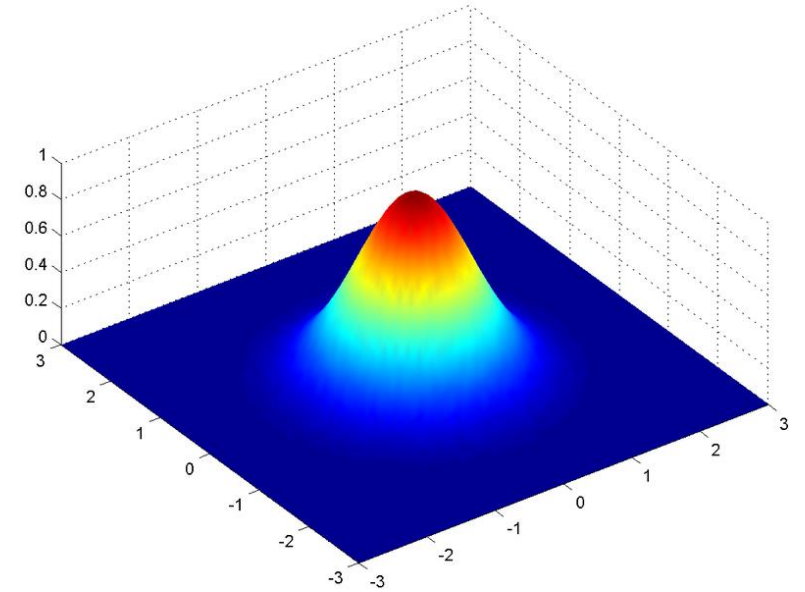
Remember our sampling strategy for Gaussian?

- The Gaussian Distribution

- $p(x \mid \mu, \sigma) = \mathcal{N}(\mu, \sigma)(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$

- Sampling:

- [Sample from the unit normal distribution](#) $\rightarrow r \sim \mathcal{N}(0, 1)$
 - Return $\mu + r\sigma$



The Reparameterization Trick

A nice property of Gaussian distributions: if we sample $z \sim \mathcal{N}(\mu, \sigma)$ we can rewrite it as:

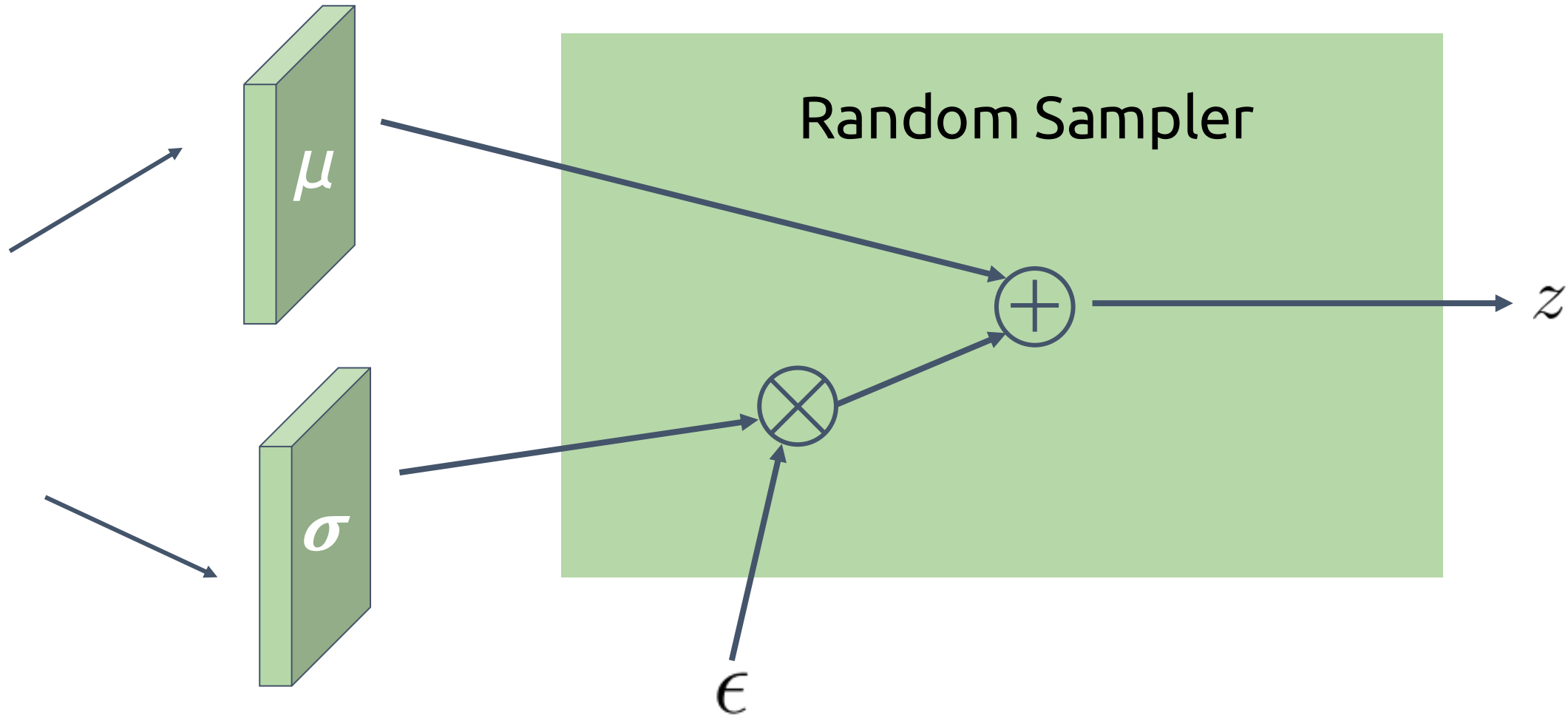
$$z = \mu + \epsilon \cdot \sigma$$

Where

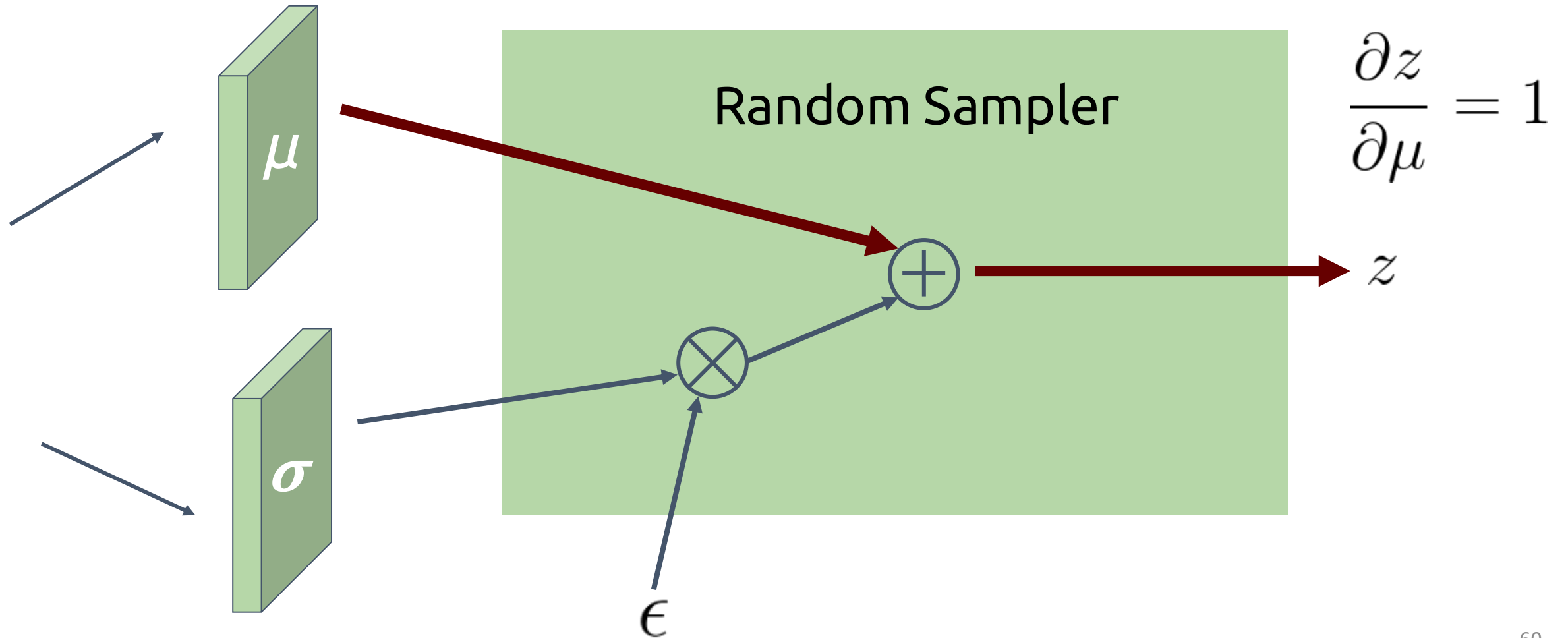
$$\epsilon \sim \mathcal{N}(0, 1)$$

- The random sampling no longer depends on learnable parameters
- This allows us to do backpropagation

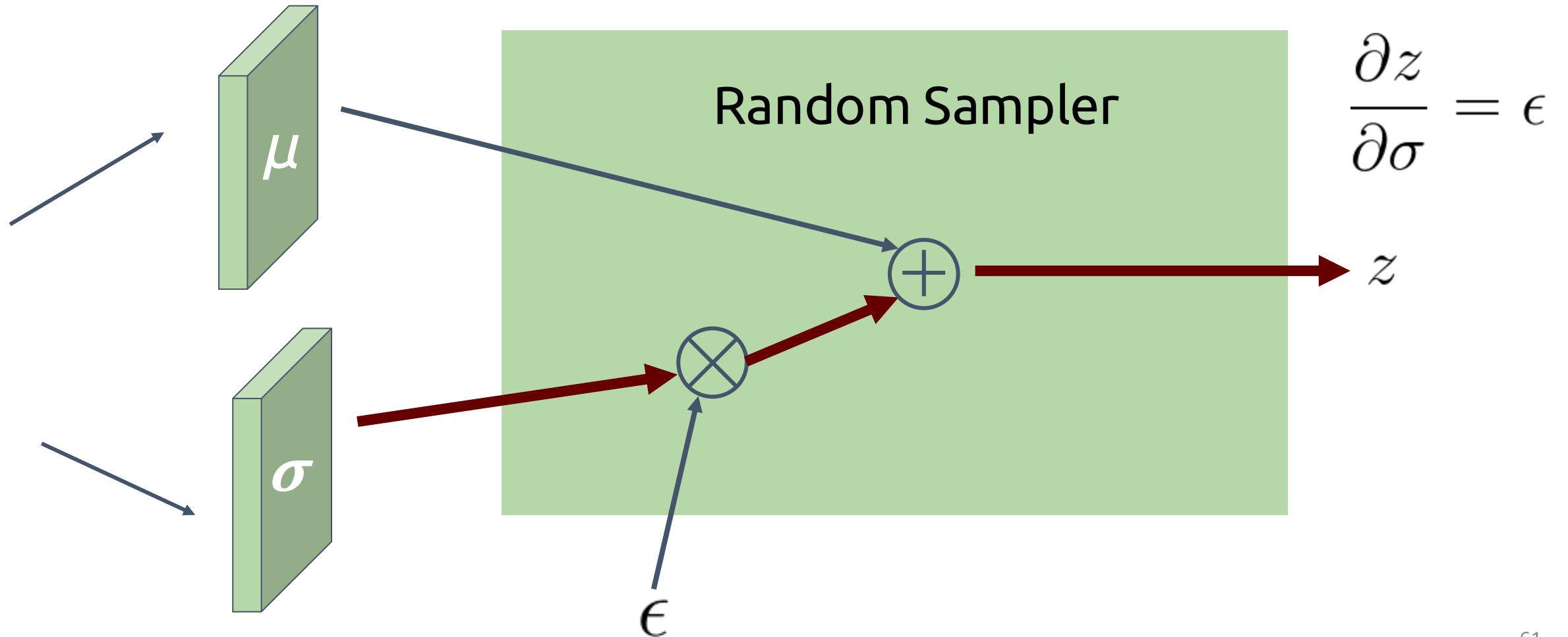
Random Sampler with Reparameterization Trick



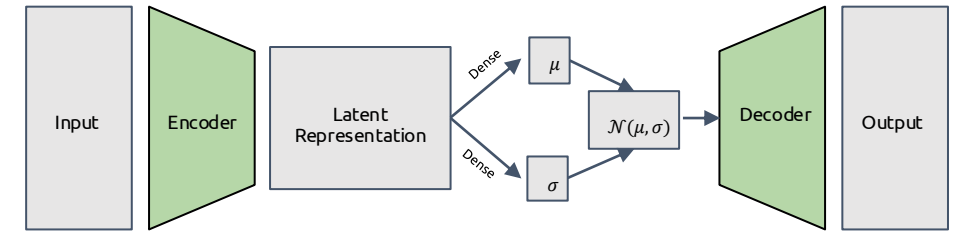
Random Sampler with Reparameterization Trick



Random Sampler with Reparameterization Trick



One more practical detail



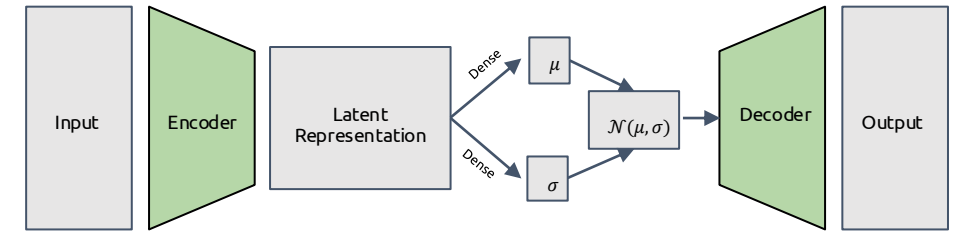
Let's again consider our sampling operation

$$z \sim \mathcal{N}(\mu, \sigma)$$

$\mu_i \in [-\infty, \infty]$ $\sigma_i \in [0, \infty]$

- Nothing prevents the neural network from outputting ***negative*** values for the standard deviation.
- Instead of predicting σ , we will instead predict $\log(\sigma^2)$. This ensures that every $\sigma_i \in [0, \infty]$
 - i.e. just treat the output of the Dense layer as if it is $\log(\sigma^2)$

One more practical detail



Let's again consider our sampling operation

$$z \sim \mathcal{N}(\mu, \sigma)$$

$\swarrow \quad \nwarrow$

$$\mu_i \in [-\infty, \infty] \qquad \sigma_i \in [0, \infty]$$

Any questions?

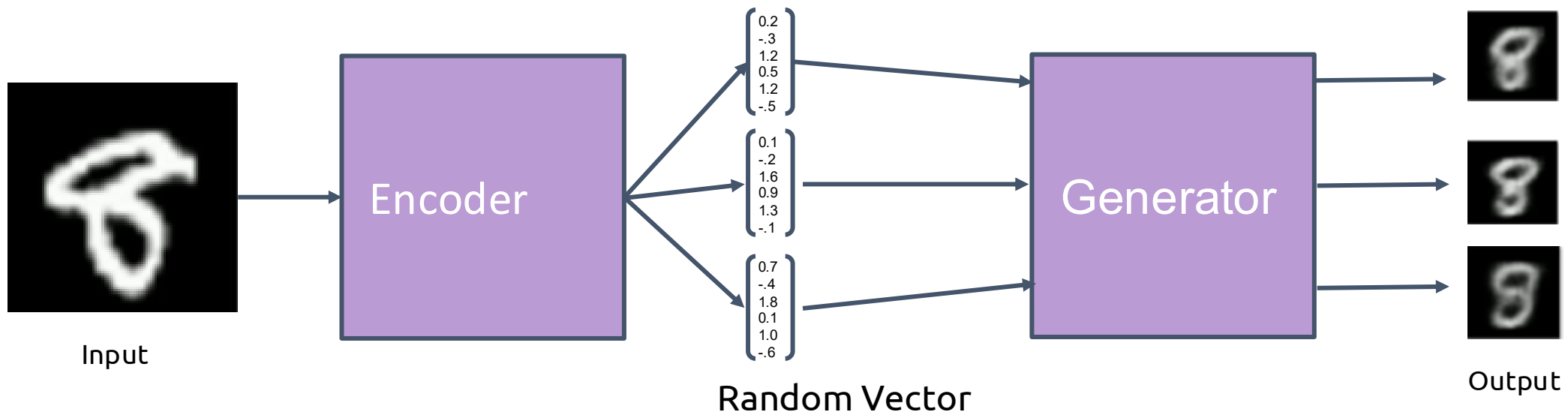


- Instead of predicting σ , we will instead predict $\log(\sigma^2)$. This ensures that every $\sigma_i \in [0, \infty]$
 - i.e. just treat the output of the Dense layer as if it is $\log(\sigma^2)$

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

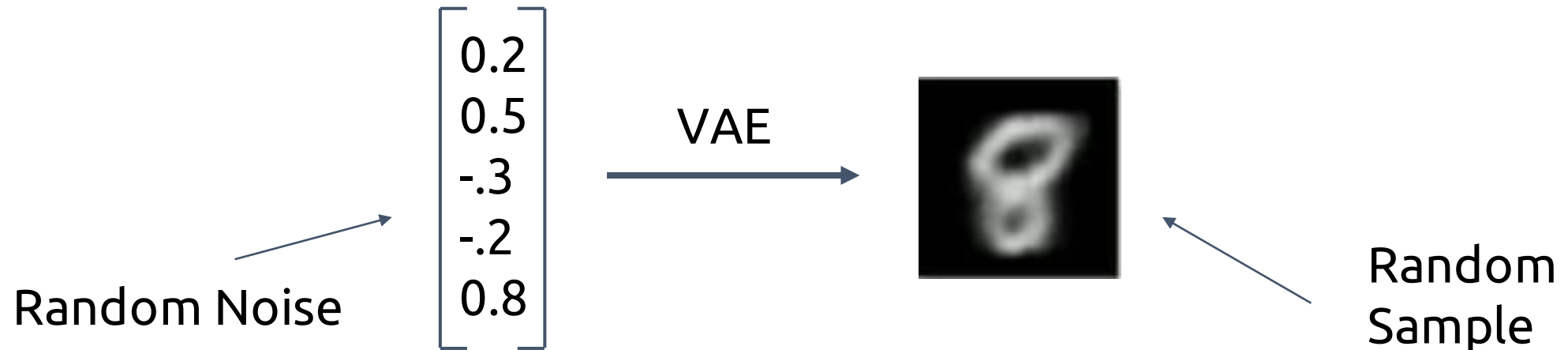
Sampling from a VAE

- We can use a trained VAE to generate random variants of an input data point...



Sampling from a VAE

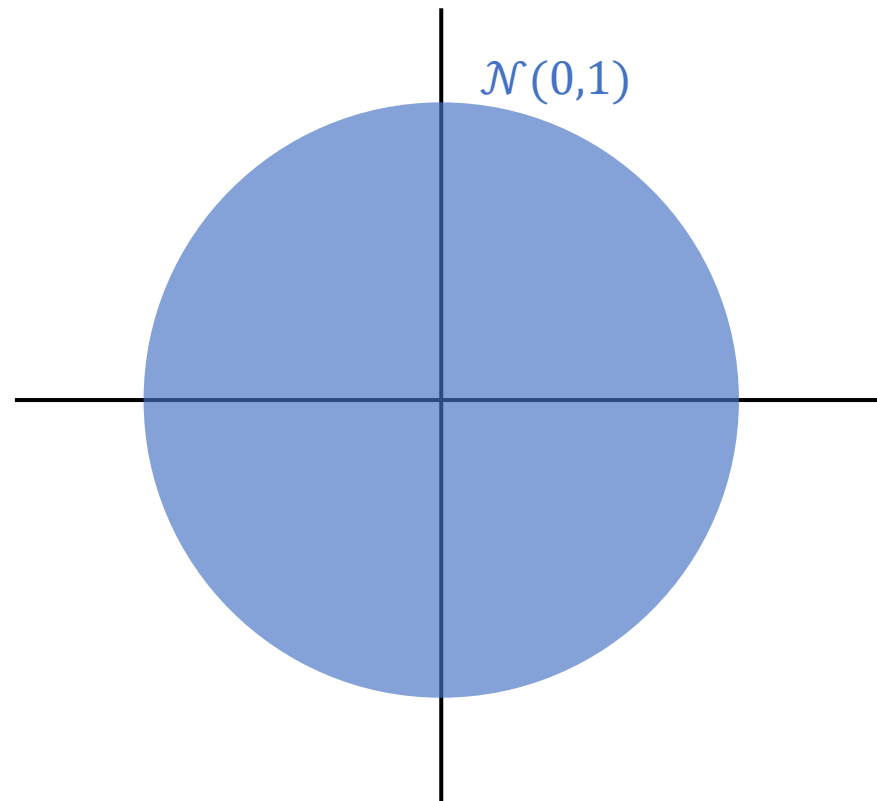
... But ultimately, we want to draw random samples from a VAE



How can we do this?

This is where our particular choice of training loss will pay off

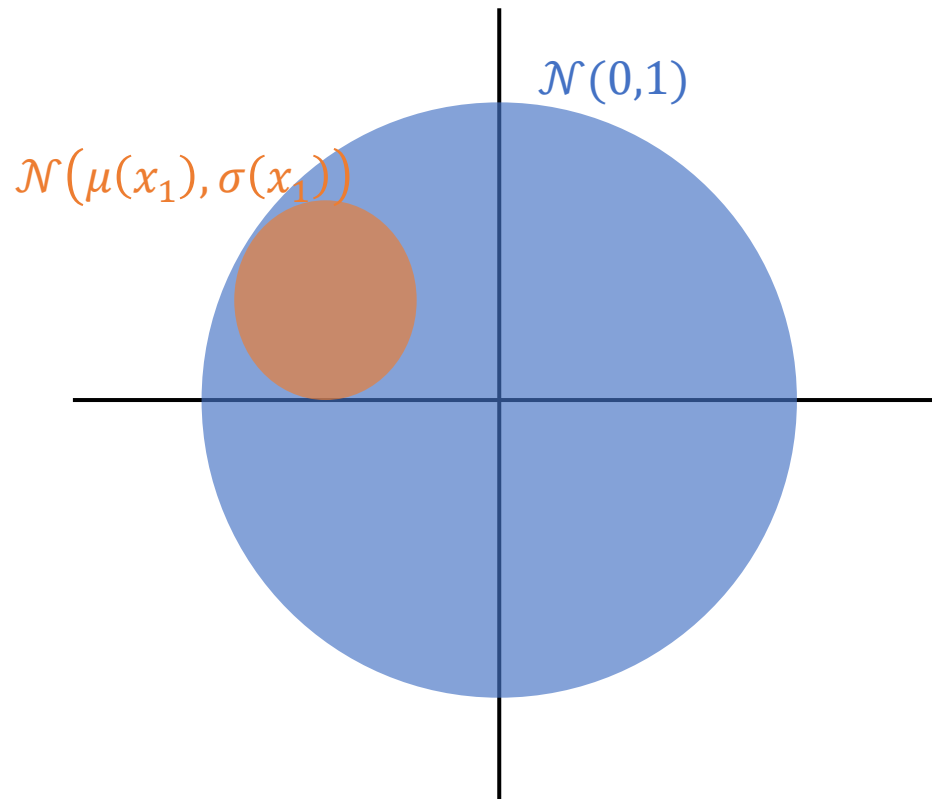
Encoding different points into latent space



Let this circle represent the probability density of a unit Gaussian in latent space

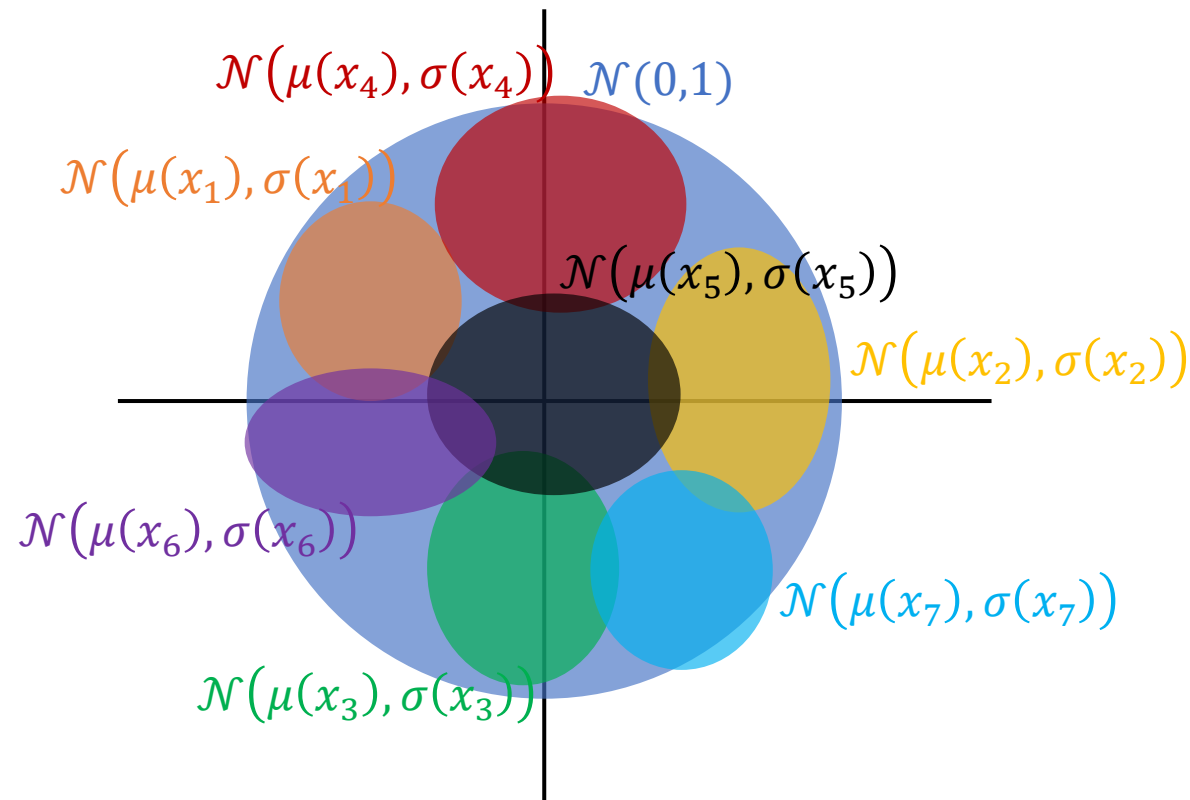
Encoding different points into latent space

Let this circle represent the probability density of the $\mathcal{N}(\mu, \sigma)$ distribution that the encoder predicts given an input data point x_1



Encoding different points into latent space

$$L = ||x - \hat{x}||_2^2 + \lambda D_{KL}(\mathcal{N}(\mu, \sigma), \mathcal{N}(0, 1))$$



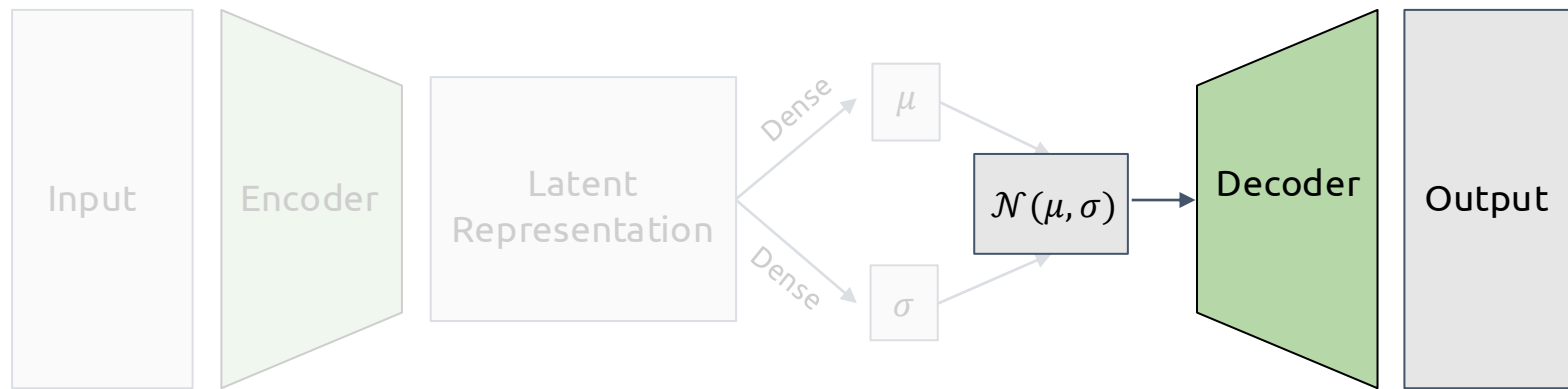
Because of our KL divergence loss, the $\mathcal{N}(\mu, \sigma)$ for any input data point has to be somewhat similar to $\mathcal{N}(0,1)$

So, if we sample a point from $\mathcal{N}(0,1)$, it is very likely to fall within one of these encoded distributions from the training set...

...which the decoder has been trained to reconstruct well!

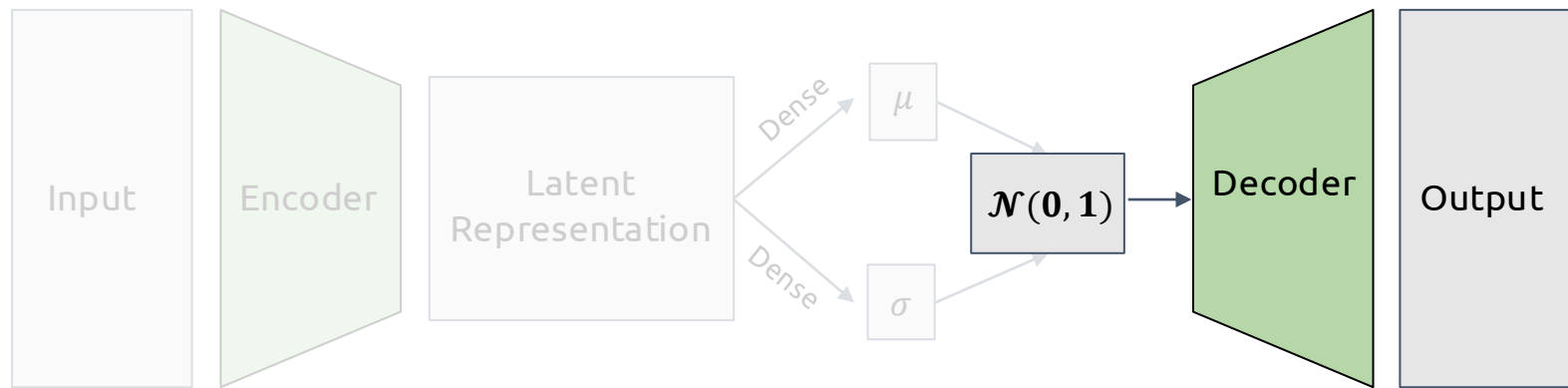
Sampling from a VAE

So what do we do?



- Discard this part of the network...

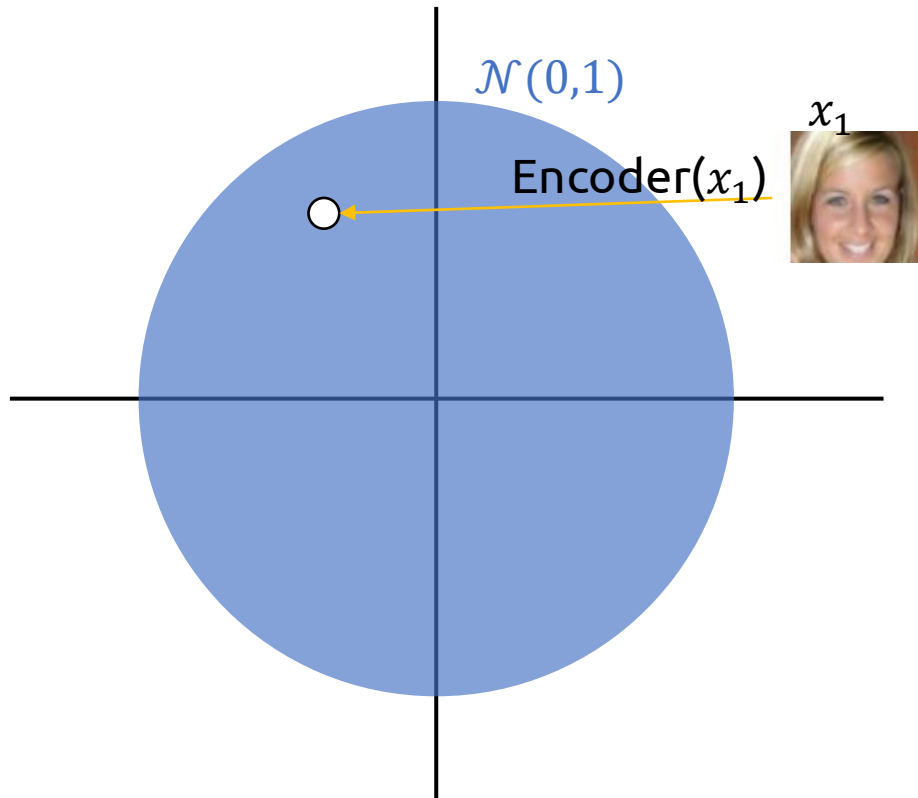
Sampling from a VAE



- Discard this part of the network...
- ...and set $(\mu, \sigma) = (0, 1)$

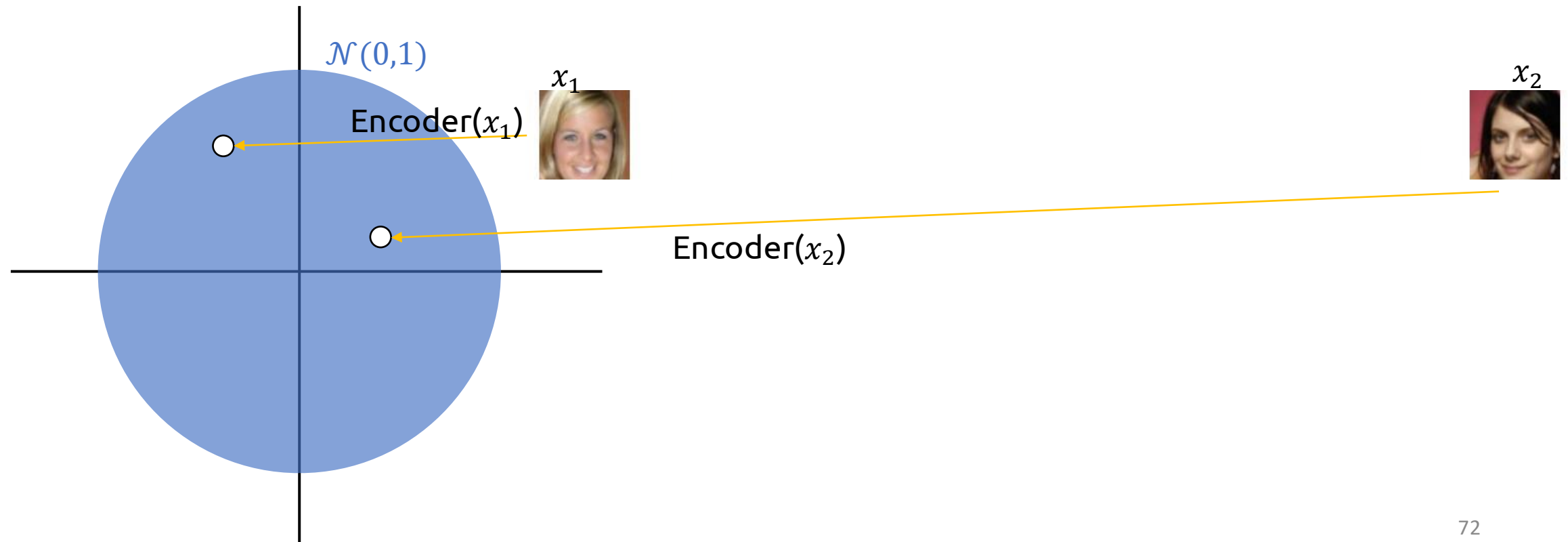
Latent Space Interpolation

- Trace a linear path between two points in latent space, put all points along the path into the decoder



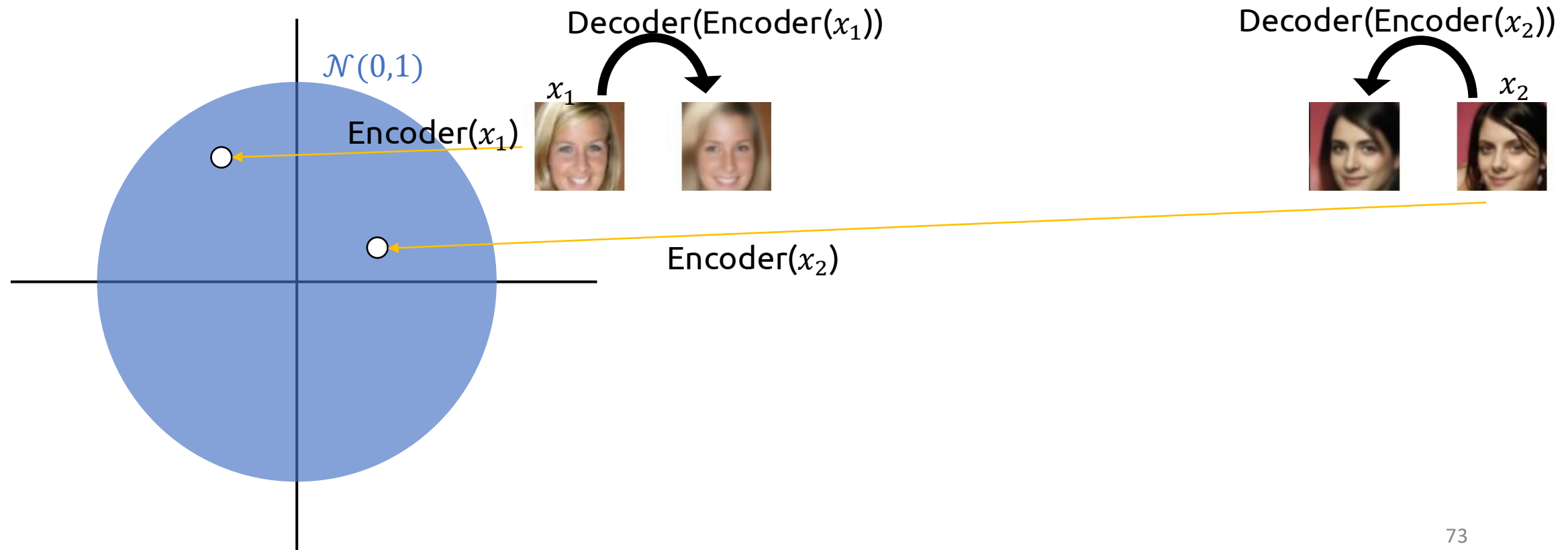
Latent Space Interpolation

- Trace a linear path between two points in latent space, put all points along the path into the decoder



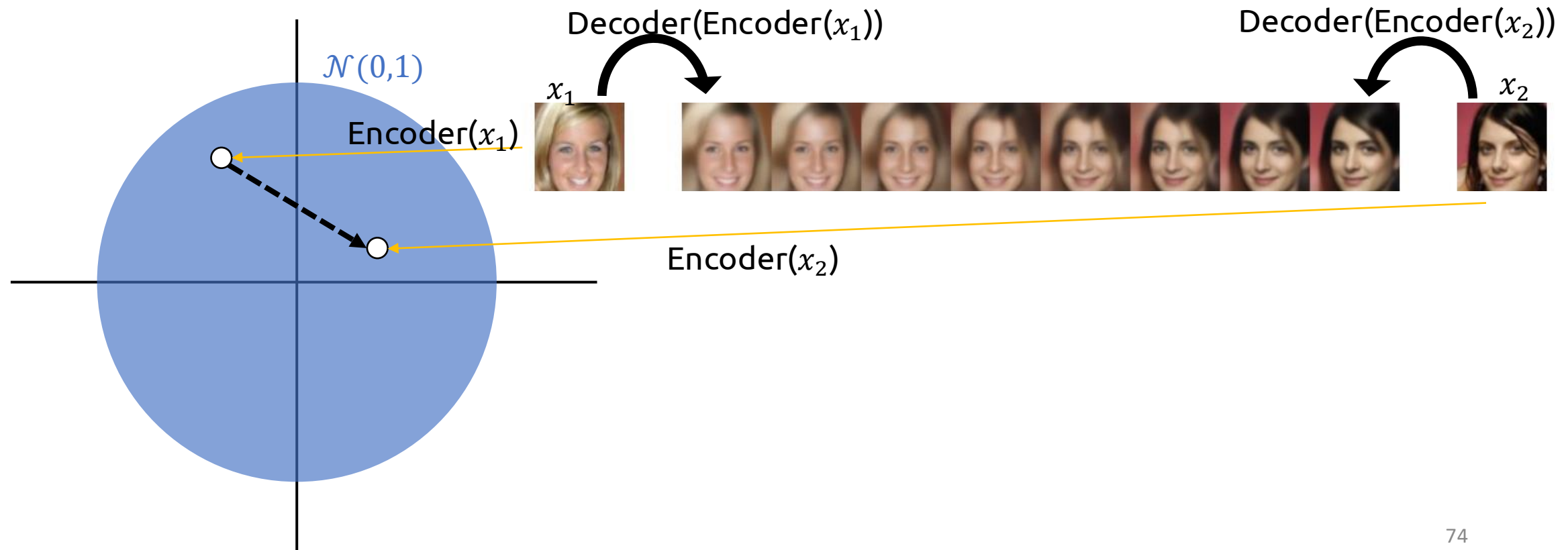
Latent Space Interpolation

- Trace a linear path between two points in latent space, put all points along the path into the decoder



Latent Space Interpolation

- Trace a linear path between two points in latent space, put all points along the path into the decoder



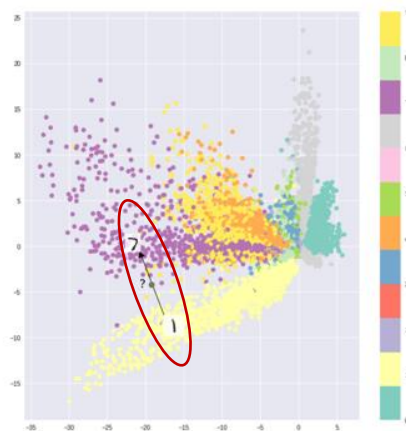


Latent Space Interpolation

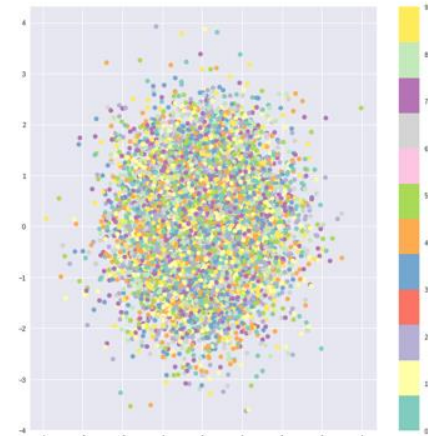
- Can also try it with a regular autoencoder
 - Doesn't work as well
 - **Why not?**
 - The KL divergence loss regularizes the shape of the latent space. Without it, a regular autoencoder might have “empty” pockets of latent space

Linear interpolation has to cross a pocket of empty space (where the behavior of the decoder is not well defined)

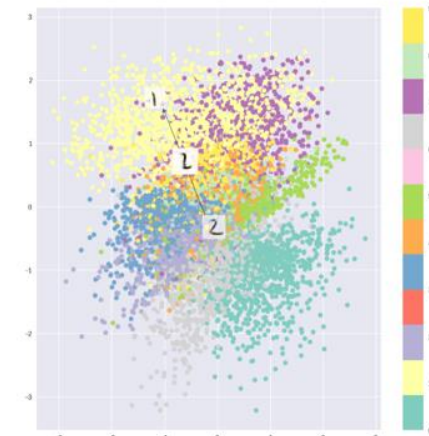
Only reconstruction loss



Only KL divergence



Combination



Conditional VAE

Discriminative vs Generative Models

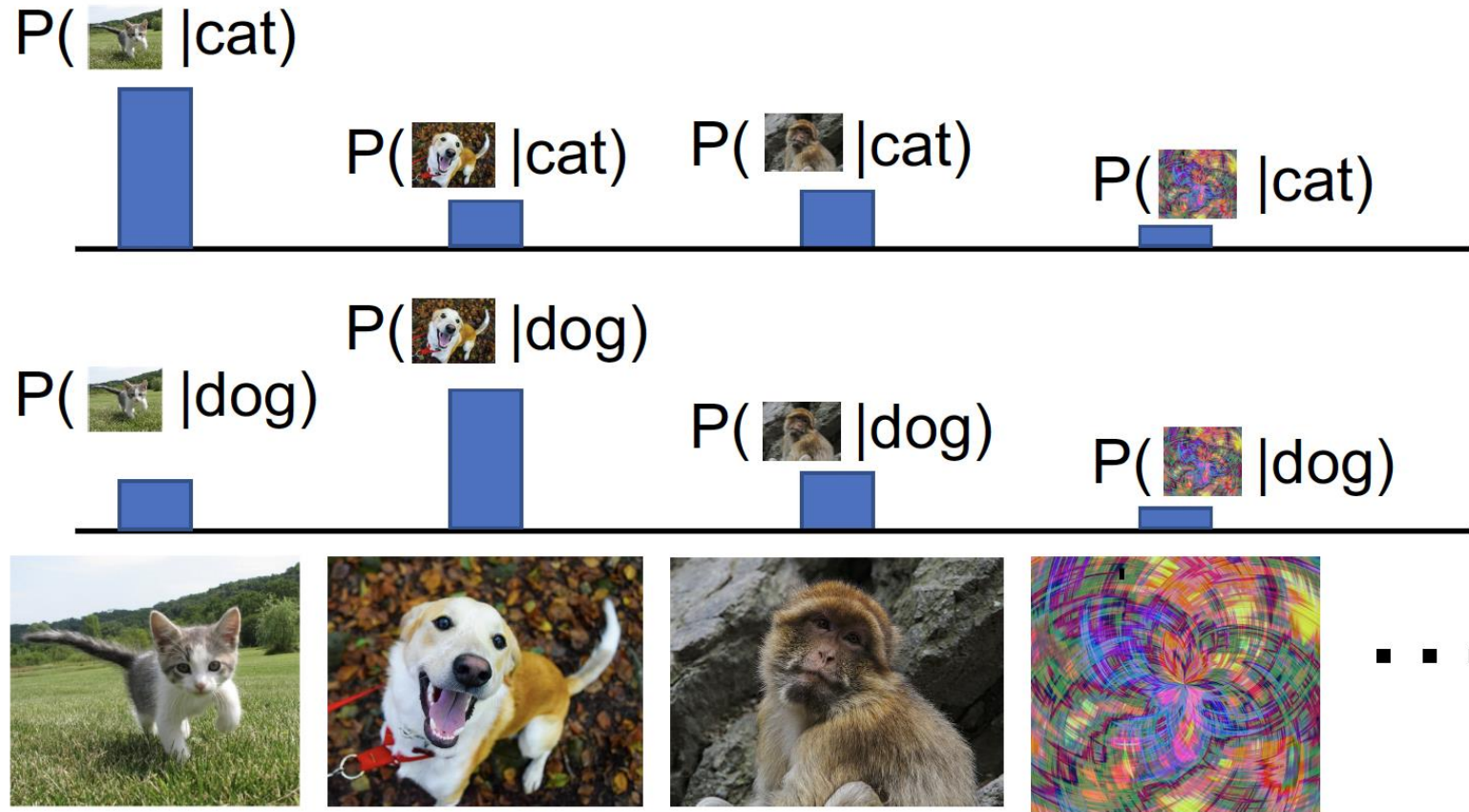
Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$

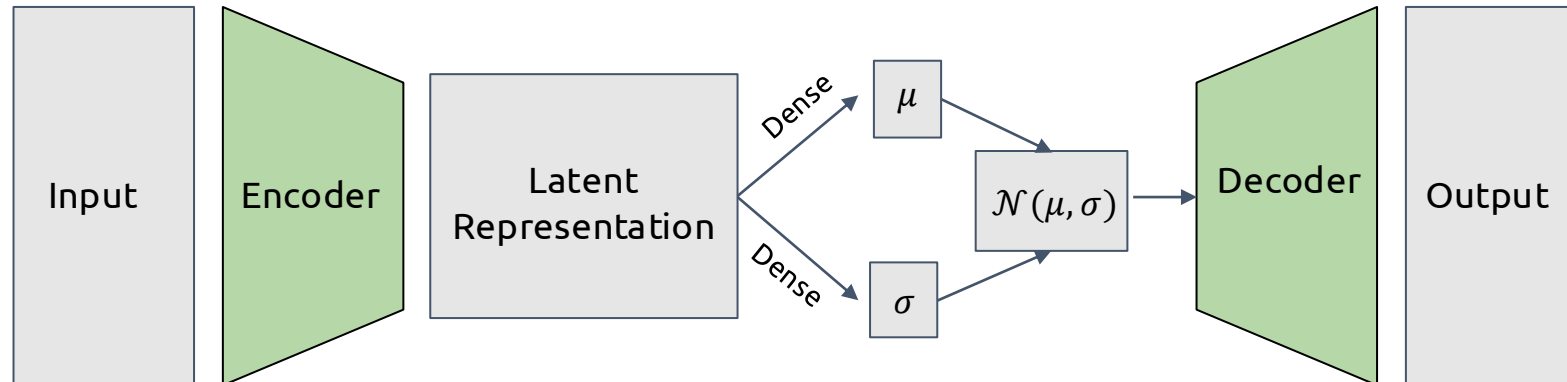
Conditional Generative Model: Learn $p(x|y)$



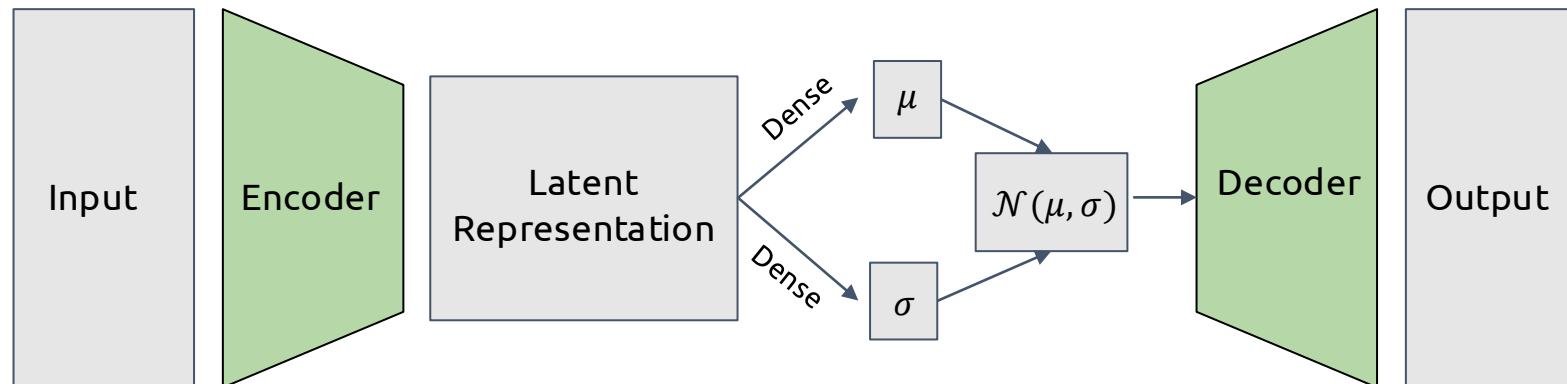
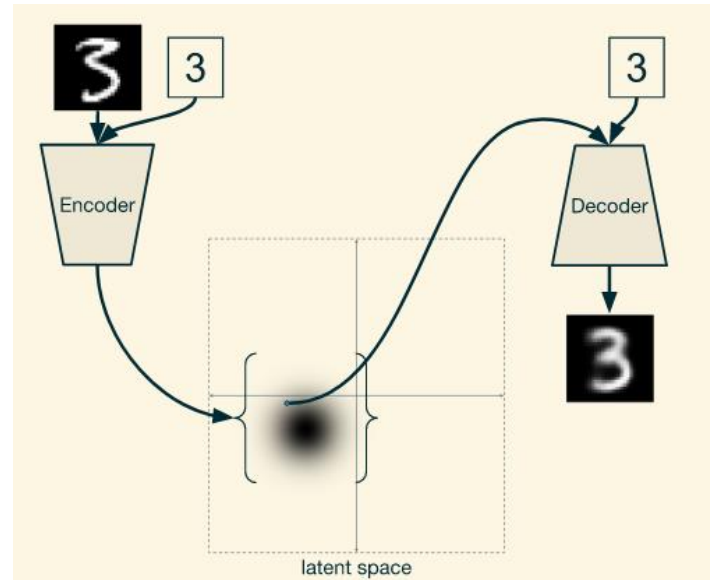
Conditional Generative Model: Each possible label induces a competition among all images

Conditional VAE

Any ideas?



Conditional VAE

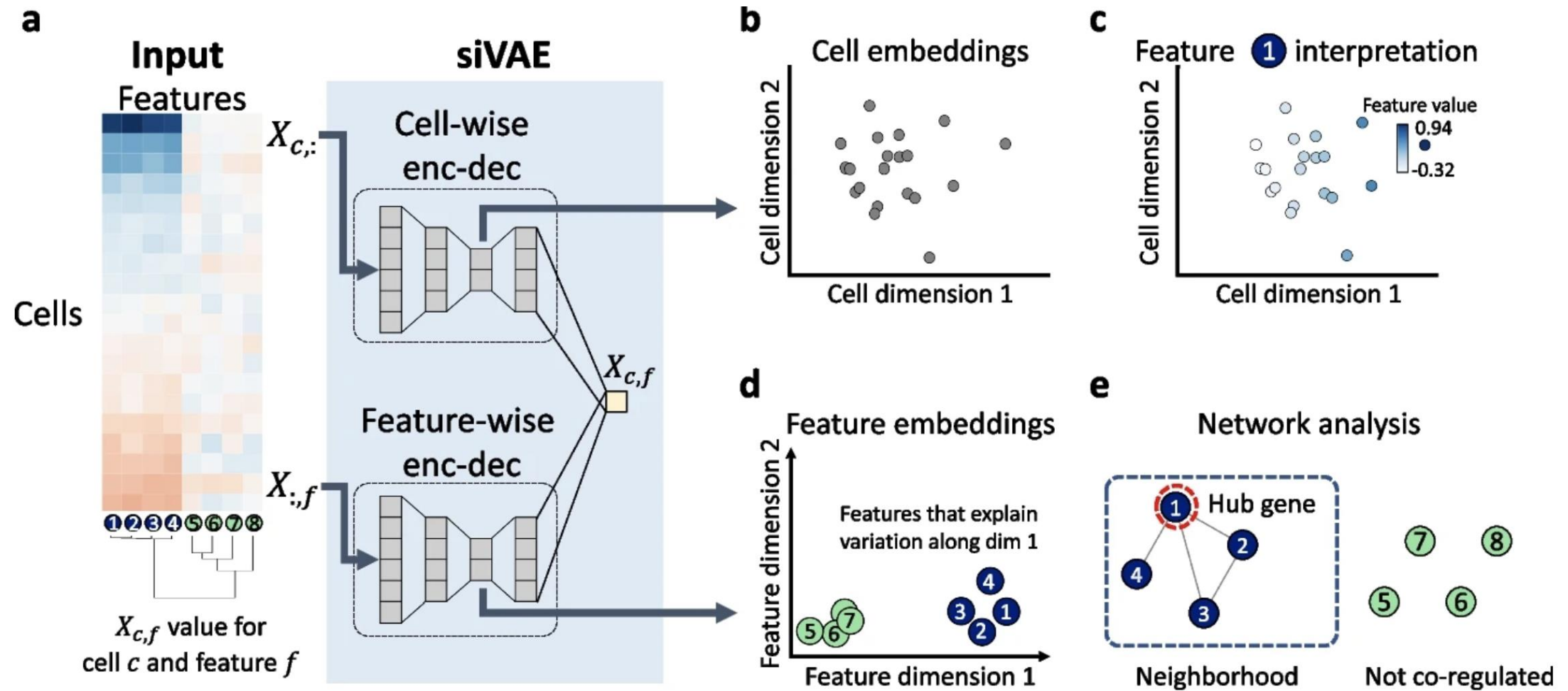


Class activity: Designing AEs/VAEs for single-cell datasets

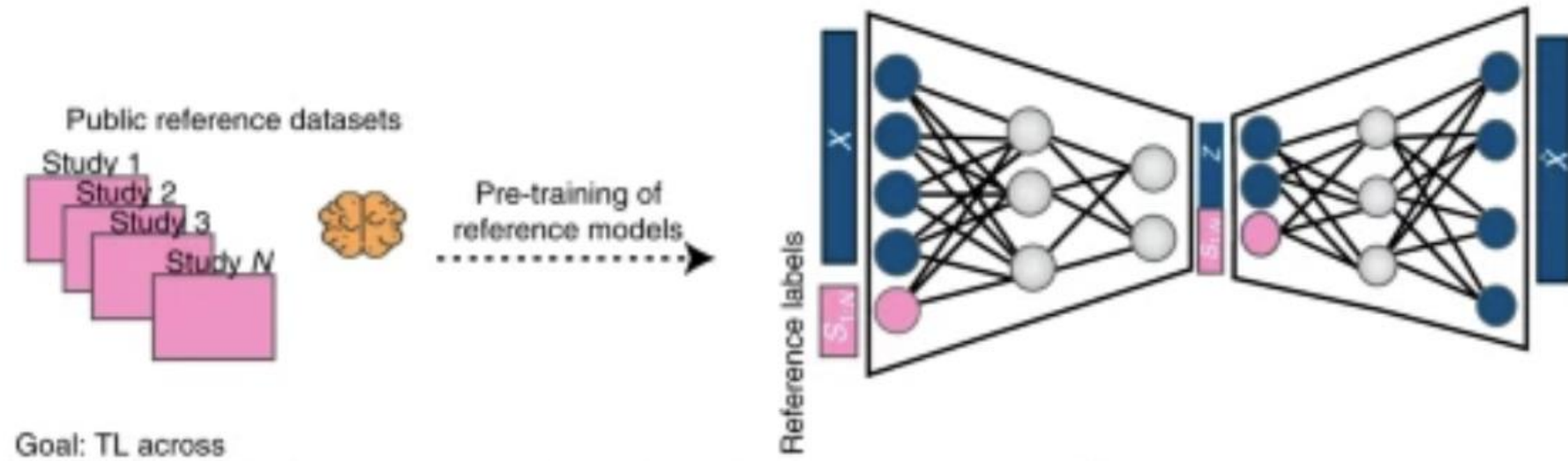
	Cell1	Cell2	...	CellN
Gene1	3	2	.	13
Gene2	2	3	.	1
Gene3	1	14	.	18
...	.	.	.	.
...	.	.	.	.
...	.	.	.	.
GeneM	25	0	.	0

Help your colleague design a deep learning model for this task:

- (1) How will you input the data?
- (2) Design an autoencoder for the task.
- (3) Design a VAE for the task.
- (4) Suppose you had to design a Conditional VAE – what could be added as a condition?



scArches enables iterative query-to-reference single-cell integration.



Recap – Learn about Variational Autoencoders (VAEs)

(1) Generative modeling

(2) VAE Architecture

(3) VAE Loss

(4) VAE considerations

(5) Conditional VAE

(6) Class activity: Designing AEs/VAEs for single-cell datasets